

ШИНЭ МОНГОЛ ТЕХНОЛОГИЙН КОЛЛЕЖ

КОМПЬЮТЕРЫН УХААНЫ ИНЖЕНЕРИЙН ТЭНХИМ

Оюутны код: s21c076b
Оюутны овог нэр: Батцэнгэл АНАР

RAG технологид тулгуурласан локал файл хайлтын ухаалаг туслагч

Intelligent Assistant for Local File Search Based on RAG Technology

/ТӨГСӨЛТИЙН СУДАЛГААНЫ АЖИЛ/

Удирдагч багш	Н.Соронзонболд
Удирдагчийн зэрэг цол	
Гүйцэтгэсэн оюутан	Б.Анар

Улаанбаатар хот
2025 он

ШИНЭ МОНГОЛ ТЕХНОЛОГИЙН КОЛЛЕЖ КОМПЬЮТЕРЫН
УХААНЫ ИНЖЕНЕРИЙН ТЭНХИМ Төгсөлтийн судалгааны ажил Оюутны код:

s21c076b

Оюутны овог нэр: Батцэнгэл АНАР **RAG** технологид тулгуурласан локал файл

хайлтын ухаалаг туслагч
Intelligent Assistant for Local File Search Based on RAG Technology

Гүйцэтгэгч: Б.Анар
Удирдагч: Н.Соронзонболд Улаанбаатар хот
Удирдагчийн зэрэг цол: _____

2025 он

Хураангуй

Энэхүү төгсөлтийн судалгааны ажлын зорилго нь интернэт холболтгүй сүлжээгүй орчинд компьютерын дотоод дискэнд хадгалагдсан баримт бичгүүдээс хиймэл оюун ухаанд суурилсан RAG (Retrieval-Augmented Generation) аргаар мэдээлэл хайх систем боловсруулах явдал юм.

Орчин үед мэдээлэл асар хурдтай өсч байгаа боловч байгууллагын дотоод мэдээллийг үр дүнтэй ашиглахад хүндрэл гардаг. Иймд энэхүү судалгаагаар дотоод орчинд ажиллах, өгөгдлийн аюулгүй байдлыг хамгаалсан хиймэл оюун ухааны хайлтын систем боловсруулах замаар уг асуудлыг шийдвэрлэхийг зорьсон.

Төслийн хүрээнд RAG архитектур ашигласан бөгөөд гурван хайлтын аргыг хослуулан ажилладаг: FAISS вектор хайлт (утгачилсан хайлт), BM25 статистик хайлт (түлхүүр үгээр хайлт), мөн шууд тохирол (үгийн тааруулалт). Бичвэрүүдийг Embedding загварт оруулж вектор болгон хувиргаж, ONNX INT8 форматаар CPU дээр хурдан ажиллах боломжийг хангасан. Хэрэглэгчийн интерфэйсийг Gradio сан ашиглан вэб хэлбэрээр хэрэгжүүлсэн бөгөөд Ollama платформоор дамжуулан Qwen2.5 загвараас хариулт үүсгэн авдаг.

Систем нь PDF, DOCX, CSV, XLSX, TXT зэрэг 10 төрлийн файлыг дэмждэг бөгөөд өгөгдлийн нууцлалыг бүрэн хангасан офлайн шийдэл юм.

Түлхүүр үг: RAG, FAISS, BM25, Hybrid хайлт, ONNX, офлайн AI, Ollama, Gradio

Abstract

The aim of this thesis research is to develop an artificial intelligence-based Retrieval-Augmented Generation (RAG) system for searching information from documents stored on a computer's local disk in an offline, network-free environment.

Although information is growing rapidly in the modern era, organizations often face difficulties in effectively utilizing their internal data. Therefore, this study aims to address this issue by developing a secure, on-premise AI-based search system that operates in a local environment while ensuring data security.

Within the scope of the project, a RAG architecture is implemented, combining three search methods: FAISS vector search (semantic search), BM25 statistical search (keyword-based search), and direct matching (exact word matching). Documents are converted into vector representations using an embedding model, and ONNX INT8 format is used to ensure fast CPU-based execution. The user interface is implemented as a web application using the Gradio library, and responses are generated from the Qwen2.5 model via the Ollama platform.

The system supports 10 types of files including PDF, DOCX, CSV, XLSX, and TXT, and is a fully offline solution that ensures data confidentiality.

Keywords: RAG, FAISS, BM25, hybrid search, ONNX, offline AI, Ollama, Gradio

Агуулга

Хураангуй	i
Abstract	ii
Товчилсон үгийн жагсаалт	v
Хүснэгтийн жагсаалт	vi
Зургийн жагсаалт	vii
1 Ажлын төлөвлөгөө	1
2 Удиртгал	2
2.1 Үндэслэл, ач холбогдол	2
2.2 Зорилго, зорилт	2
2.3 Судалгааны хязгаарлалт	2
3 Судалгааны сэдвийн онол, өнөөгийн түвшин	3
3.1 RAG аргачлалын үндэс	3
3.2 Хосолмол хайлтын онол	4
3.3 Embedding (вектор дүрслэл) онол	5
3.4 Transformer архитектур ба Attention механизм	5
3.5 ONNX ба загварын оновчлолын онол	6
3.6 Мэдээлэл хайлтын үнэлгээний үзүүлэлтүүд	7
3.7 Ижил төрлийн судалгааны ажлуудын харьцуулалт	8
4 Судалгааны арга зүй	9
4.1 Системийн ерөнхий архитектур	9
4.2 Системийн ажиллах дараалал	9
4.3 Файл уншигч модуль	11
4.4 Chunking (хэсэгчлэл)	13
4.5 Embedding ба ONNX оновчлол	13
4.6 Embedding кэш систем	15
4.7 FAISS вектор индекс	15
4.8 BM25 статистик индекс	15
4.9 Холимог хайлтын хэрэгжүүлэлт	16
4.10 Асуулт өргөтгөлт (Query Expansion)	18
4.11 LLM-д тулгуурласан хайлт сайжруулалт	19
4.12 Reranker	20
4.13 LLM хариулт үүсгэгч (Ollama)	20
4.14 Хэрэглэгчийн дэлгэц (Gradio)	21
4.15 Хөгжүүлэлтийн явцад тулгарсан хүндрэлүүд ба шийдлүүд	22
5 Судалгааны үр дүн, дүгнэлт	24
5.1 Гүйцэтгэлийн хэмжилт	24
5.2 Индексэлэлтийн задаргаа	24
5.3 Оновчлолын үр дүн	25

5.4	Хайлтын нарийвчлалын харьцуулалт	25
5.5	Query Expansion-ийн үр нөлөө	26
5.6	Файлын форматын дэмжлэг	26
5.7	Дүгнэлт	27
Ном зүй		29
Талархал		31
Хавсралт		32

Товчилсон үгийн жагсаалт

1. **RAG** – Retrieval-Augmented Generation — Хайлтад суурилсан хариулт үүсгэх арга
2. **FAISS** – Facebook AI Similarity Search — Вектор хайлтын сан
3. **BM25** – Best Matching 25 — Статистик түлхүүр үгийн хайлтын алгоритм
4. **ONNX** – Open Neural Network Exchange — Нейрон сүлжээний загвар солилцох стандарт
5. **INT8** – 8-bit Integer Quantization — Загварын хэмжээг багасгах арга
6. **LLM** – Large Language Model — Том хэлний загвар
7. **Embedding** – Бичвэрийг тоон вектор хэлбэрт хөрвүүлэх
8. **UI** – User Interface — Хэрэглэгчийн интерфэйс
9. **PDF** – Portable Document Format — Баримт бичгийн стандарт формат
10. **API** – Application Programming Interface — Програм хооронд харилцах холболт
11. **Chunk** – Баримтыг жижиг хэсгүүдэд хуваасан нэгж
12. **IVF** – Inverted File Index — FAISS-ийн хурдан хайлтын бүтэц
13. **IDF** – Inverse Document Frequency — Баримт дахь үгийн ховор байдлын хэмжүүр
14. **JSON** – JavaScript Object Notation — Өгөгдөл солилцох стандарт бүтэцтэй формат
15. **VSCode** – Visual Studio Code — Кодыг ажлуулдаг өргөтгөлтэй программ

Хүснэгтийн жагсаалт

1	Судалгааны ажлын төлөвлөгөө	1
2	Ижил төрлийн системүүдийн харьцуулалт	8
3	Дэмжигдэх файлын төрлүүд ба уншигч	11
4	Embedding загварууд	14
5	Хайлтын горим бүрийн жинлэлт	16
6	Confidence score тооцооллын бүрэлдэхүүн	18
7	LLM загваруудын харьцуулалт	20
8	Системийн бодит гүйцэтгэлийн хэмжилт (CPU горим, Intel OpenVINO)	24
9	350 MB баримтын санг индексэлэхэд зарцуулсан хугацааны задаргаа	24
10	Оновчлол тус бүрийн хэмжилтийн харьцуулалт	25
11	Хайлтын аргын нарийвчлалын харьцуулалт (50 асуулт, Top- $k=5$)	25
12	Query Expansion өргөтгөлийн өмнөх болон дараах олдсон chunk-ийн тоо	26
13	Файлын формат тус бүрийн уншилтын үр дүн (98 файл, нийт)	26

Зургийн жагсаалт

1	RAG-ийн үндсэн бүтэц	3
2	Системийн өгөгдлийн урсгал (индексжүүлэлт ба хайлт)	9
3	Системийн ажиллах дарааллын ерөнхий зураглал	10
4	Асуултын ангилал ба жин сонголтын урсгал	17
5	RAG системийн ажлын дараалал: эхлүүлэх, индексэлэх, асуулт-хариултын урсгал	21

1 Ажлын төлөвлөгөө

Хүснэгт 1: Судалгааны ажлын төлөвлөгөө

№	Төлөвлөгөө	Эхлэх	Дуусах	Явц
1	Онол, арга зүйн материалуудыг судлах	2025.10.15	2025.10.31	
2	Системийн ерөнхий төлөвлөгөө боловсруулах	2025.11.1	2025.11.15	
3	Диск доторх хайлт болон файлуудыг судлах	2025.11.15	2025.11.30	
4	ТСА үзлэг 1	2025.12.2		
5	Файл уншигч модуль хөгжүүлэх, турших	2025.12.3	2025.12.15	
6	Embedding, FAISS индекс хэрэгжүүлэх	2025.12.15	2025.12.28	
7	BM25 + Keyword хайлт нэмж Hybrid болгох	2026.1.2	2026.1.15	
8	Gradio UI хөгжүүлэх, Ollama холбох	2026.1.15	2026.1.31	
9	ONNX INT8 оновчлол, кэш систем	2026.2.1	2026.2.15	
10	Туршилтын демо бэлтгэх	2026.2.15	2026.2.28	
11	ТСА үзлэг 2	2026.3.1		
12	4000 файлын оновчлол, хурдасгал	2026.3.2	2026.3.15	
13	Хэрэглэгчийн туршилт	2026.3.15	2026.3.31	
14	Судалгааны бичвэрийг боловсруулж дуусгах	2026.4.1	2026.4.15	
15	Туршилтын үр дүнг нэгтгэх	2026.4.15	2026.4.29	
16	ТСА урьдчилсан хамгаалалт	2026.4.30		
17	Сайжруулалт хийх	2026.5.1	2026.5.15	
18	ТСА хамгаалалтанд бэлэн болох	2026.5.15	2026.5.30	
19	ТСА үндсэн хамгаалалт	2026.5.31		

2 Удиртгал

2.1 Үндэслэл, ач холбогдол

Орчин үед мэдээллийн хэмжээ хурдацтай өсч, байгууллага болон хувь хүний компьютерт олон төрлийн их хэмжээний өгөгдлийг богино хугацаанд боловсруулах, хайх, дүн шинжилгээ хийх шаардлага улам нэмэгдэж байна. Гэвч компьютерын дотоод дискэнд хадгалагдсан файлуудаас шаардлагатай мэдээллийг гараар хайх нь цаг их шаарддаг, алдаа гарах магадлал өндөр үйл явц юм.

APQC-ийн 2023 оны KM Benchmark судалгаагаар мэдлэгийн ажилтнууд ажлынхаа 20–30%-ийг зөвхөн мэдээлэл хайх эсвэл дахин бүтээхэд зарцуулдаг бөгөөд энэ нь өдөрт дунджаар 1.8–2 цаг орчим юм. Хайлтын систем сайн ажилладаг байгууллагуудад энэ хугацааг 50% хүртэл багасгаж чаддаг гэж тооцдог.

Мөн ихэнх RAG системүүд (ChatGPT, Perplexity гэх мэт) нь интернэт холболт шаарддаг тул байгууллагын нууц мэдээллийг гадаад серверт илгээх аюулгүй байдлын эрсдэлтэй. Иймээс бүрэн офлайн орчинд ажилладаг, өгөгдлийн нууцлалыг хамгаалсан RAG систем боловсруулах шаардлагатай гэж үзсэн.

2.2 Зорилго, зорилт

Энэ судалгааны зорилго нь компьютерын дотоод дискэнд хадгалагдсан бичвэрийн өгөгдлийг боловсруулан, хэрэглэгчийн асуултанд тохирох мэдээллийг автоматаар илрүүлж, холбогдох хариултыг гаргаж чаддаг RAG технологид суурилсан хариултын системийг боловсруулан ажлын цаг хэмнэх, үр бүтээмжтэй байдлыг нэмэгдүүлэх юм.

Зорилт:

- Локал дискнээс бичвэрийн төрлийн файл уншиж, вектор хэлбэрт хөрвүүлэх процессыг хэрэгжүүлэх
- FAISS вектор хайлт, BM25 статистик хайлт, keyword matching гэсэн гурван аргыг нэгтгэсэн Hybrid хайлт бүтээх
- Хэрэглэгчийн асуултанд үндэслэн утгачилсан хариулт үүсгэх LLM загвар ажлуулах
- ONNX INT8 оновчлол ашиглан CPU дээр хурдан ажиллах боломжтой болгох
- Хайлтын системийг туршиж оновчтой байдлыг нэмэгдүүлэх
- Gradio ашиглан хэрэглэгчид хялбар вэб интерфейс хэрэгжүүлэх

2.3 Судалгааны хязгаарлалт

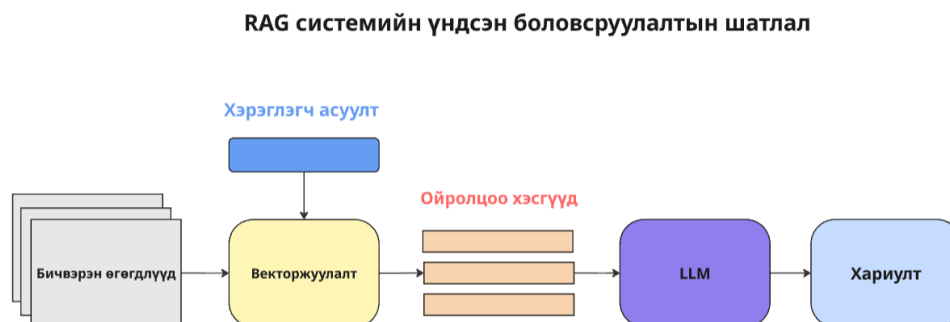
Систем нь дотоод орчинд бүрэн ажилладаг боловч дараах хязгаарлалттай:

- Зөвхөн бичвэрт суурилсан файлуудыг дэмждэг — зураг, аудио, видео файл уншиж чадахгүй
- Зурган PDF (scan хийсэн) файлаас текст гаргахад OCR технологийн шаардлагатай
- Локал LLM загварын хариултын чанар GPT-4 зэрэг том загвараас доогуур
- Монгол хэлний вектор загварын нарийвчлал хязгаарлагдмал (multilingual загвар ашиглаж байгаа)
- GPU байхгүй тохиолдолд индекс үүсгэх болон хайлтын хурд удаан байж болно

3 Судалгааны сэдвийн онол, өнөөгийн түвшин

3.1 RAG аргачлалын үндэс

RAG (Retrieval-Augmented Generation) нь хайлт болон хэлний загварын үүсгэлтийг нэгтгэдэг аргачлал юм. Уламжлалт LLM загварууд зөвхөн сургалтын үеийн мэдлэгт тулгуурладаг бол RAG нь гадаад эх сурвалжаас мэдээлэл татан авч загварт контекст болгон өгдгөөрөө давуу талтай.



Зураг 1: RAG-ийн үндсэн бүтэц

3.1.1 Retriever бүрэлдэхүүн

Retriever нь RAG-ийн “хайлт” хэсгийг хариуцдаг:

1. **Indexing үе** — Бичвэрүүдийг embedding загвараар тоон вектор болгон FAISS индексэд хадгална.
2. **Query боловсруулах үе** — Хэрэглэгчийн асуултыг мөн адил вектор болгон хөрвүүлнэ.
3. **Хайх үе** — Хоёр вектор хоорондын косинус зайгаар хамгийн ойрхон утгатай баримтыг олдог.

3.1.2 Generator (LLM) бүрэлдэхүүн

Generator нь Retriever (Мэдээлэл хайгч модуль)-ээс олж ирсэн холбогдох мэдээлэл дээр үндэслэн хүнд ойлгомжтой хариулт үүсгэдэг хэлний загвар юм. RAG технологид Generator нь зөвхөн LLM-ийн өмнө сурсан мэдлэгт тулгуурлахгүй, харин Retriever-ээс авчирсан баримт болон лавлах мэдээллийг уншиж боловсруулан хариулт гаргадаг. Иймээс бодит эх сурвалжид тулгуурласан хариулт үүсэх боломж нэмэгдэж, буруу эсвэл зохиомол мэдээлэл (hallucination) үүсэх эрсдэл мэдэгдэхүйц багасдаг.

3.1.3 Ажиллах ерөнхий дараалал

1. Баримт бичгүүдийг уншиж, chunk хэлбэрт хуваана
2. Chunk бүрийг embedding загвараар тоон вектор болгон FAISS-д хадгална
3. BM25 статистик индекс бүтээнэ
4. Хэрэглэгч асуулт оруулахад гурван аргаар зэрэг хайна
5. Хамгийн холбоотой Top-K chunk-ийг LLM-д контекст болгон өгнө
6. LLM хариулт үүсгэж хэрэглэгчид харуулна

3.2 Хосолмол хайлтын онол

Нэг хайлтын аргад найдах нь ерөнхий нөхцөлд хамгийн сайн үр дүн өгдөггүй. Вектор хайлт утгачилсан асуултанд давуу ч яг нэг үгийн хайлтанд дутмаг, харин BM25 нь эсрэг зарчмаар ажилладаг. Иймд энэ систем гурван аргыг хослуулсан **Hybrid** шийдлийг ашигласан.

3.2.1 Вектор хайлт (FAISS)

FAISS сан нь бичвэрийн утга, агуулгыг 384 хэмжээст тоон вектор болгон хувиргаж, косинусын томъёогоор векторуудын хоорондын зайг тооцоолон хамгийн ойролцоо утгатай баримтыг олдог. FAISS индексийн хоёр горим:

- **IndexFlatIP** — Бүх векторуудыг шууд харьцуулдаг. $\text{Chunk} \leq 5,000$ үед тохиромжтой, нарийвчлал өндөр.
- **IndexIVFFlat** — Векторуудыг $\sqrt{n}$ кластерт хуваан хайдаг. $\text{Chunk} > 5,000$ үед хурд эрс нэмэгддэг.

3.2.2 Статистик хайлт (BM25)

BM25 нь баримт бүрийг хэрэглэгчийн хайсан үгтэй хэр тохирч байгаагаар оноогоор эрэмбэлдэг:

$$\text{BM25}(q, d) = \sum_{t \in q} \log \left(\frac{N - n_t + 0.5}{n_t + 0.5} + 1 \right) \cdot \frac{f_{t,d} (k_1 + 1)}{f_{t,d} + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}} \right)}$$

Энд N нийт баримтын тоо, n_t тухайн үг агуулсан баримтын тоо, $k_1 = 1.5$, $b = 0.75$.

3.2.3 Шууд тэмдэгтийн хайлт (Keyword Matching)

Текст доторх үг, тэмдэгтүүдийг яг ижил дарааллаар нь харьцуулдаг энгийн боловч үр дүнтэй арга. Нэр, код, дугаар, тусгай нэр томъёо зэрэг яг ижил хэлбэрээр бичигдсэн мэдээллийг олоход тохиромжтой.

3.2.4 Гурван аргын эцсийн нэгтгэл

$$\text{Score}_{\text{total}} = w_v \times V_{\text{score}} + w_b \times B_{\text{score}} + w_k \times K_{\text{score}}$$

Энд V — FAISS вектор оноо, B — BM25 оноо, K — түлхүүр үгийн оноо.

`enhanced_search_pipeline()` функц нь асуултын агуулгыг шинжилж, түүнд тохирох жингүүд (w_v, w_b, w_k)-ийг `WEIGHT_PROFILES`-оос сонгон ашигладаг. Эдгээр жин нь векторын төстэй байдал, текстийн давтамж дээр суурилсан хайлт (BM25), мөн түлхүүр үг таарах зэрэг хүчин зүйлүүдийн нөлөөллийг тодорхойлдог.

Асуултуудыг агуулгын шинжээр нь дөрвөн төрөлд ангилдаг. Үүнд: баримтад суурилсан асуулт, утга агуулгад төвлөрсөн асуулт, яг ижил үг хэллэгийг хайх асуулт, мөн эдгээрийг хослуулсан ерөнхий асуулт. Төрлөөс нь хамааран хайлтын жинг өөрөөр тооцоолдог.

Хэрэв хэлний загвар ашиглах боломжгүй тохиолдолд нөөц (fallback) хайлтын горим ажиллана. Энэ үед `store.search()` функц нь векторын төстэй байдал (V) болон BM25 оноо (B)-г $0.45 \times V + 0.40 \times B$ харьцаагаар нэгтгэж, дээр нь түлхүүр үг таарсан эсэхээс хамаарсан нэмэлт оноо нэмж эцсийн үр дүнг гаргадаг.

3.3 Embedding (вектор дүрслэл) онол

3.3.1 Word Embedding-ийн хөгжил

Бичвэрийг тоон дүрслэлд хөрвүүлэх арга нь мэдээллийн хайлтын технологийн гол суурь болдог. Энэ салбар нь дараах үе шатаар хөгжсөн:

- **TF-IDF (1972)** — Баримт дахь үгийн давтамж болон нийт корпусд хэр ховор тохиолдохыг хосолсон анхны статистик арга. Утгачилсан ойролцоо байдлыг тооцоолж чаддаггүй.
- **Word2Vec (2013)** — Google-ийн Mikolov нарын боловсруулсан нейрон сүлжээнд суурилсан арга. “King – Man + Woman = Queen” хэлбэрийн аналогийг харилцааг вектор хэлбэрт илэрхийлэх боломжтой болсон [15].
- **BERT (2018)** — Google-ийн Transformer архитектурт суурилсан хоёр чиглэлт хэлний загвар. Үгийн утгыг контекстэд тулгуурлан тодорхойлдог тул нэг үг өөр өөр утгатай тохиолдолд зөв ялгадаг [16].
- **Sentence-BERT (2019)** — Siamese нейрон сүлжээний тусламжтайгаар бичвэрийн бүхэл өгүүлбэрийн утгыг нэг вектор болгон дүрслэх загвар. Семантик хайлтад хамгийн тохиромжтой [4].

3.3.2 Косинусын ижил төстэй байдал

Вектор хайлтад хоёр бичвэрийн ойролцоо байдлыг дараах томъёогоор хэмждэг:

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \cdot \sqrt{\sum_{i=1}^n B_i^2}}$$

Утга нь $[-1, 1]$ мужид байдаг бөгөөд 1-д ойртох тусам хоёр вектор илүү ижил утгатай гэсэн үг. Энэхүү системд FAISS-ийн METRIC_INNER_PRODUCT хэмжүүрийг хэрэглэсэн бөгөөд векторуудыг урьдчилан нормчилсон тохиолдолд энэ нь косинус ижил төстэй байдлтай тэнцүү болдог.

3.3.3 Multilingual Embedding загвар

Монгол хэлийг дэмжихийн тулд олон хэлний загварыг сонгосон. paraphrase-multilingual-`MiniLM-L12-v2` загвар нь 50 гаруй хэлний өгөгдлөөр сургагдсан бөгөөд хэл хоорондын семантик ижил байдлыг ижил вектор орчинд дүрслэдэг онцлогтой. `multilingual-e5-small` нь Wang нарын E5 аргачлалд суурилсан бөгөөд текстийг вектор хэлбэрт хувиргахдаа “query:” болон “passage:” гэсэн тусгай утгварыг ашигладаг. Энэ бүтэц нь хэрэглэгчийн асуулт болон баримт бичгийн агуулгыг нэг ижил утгын орон зайд дүрслэх боломжийг олгож, улмаар зөвхөн үгийн давхцлаас хамаарахгүйгээр утгын хувьд ижил төстэй мэдээллийг илрүүлэх нөхцөлийг бүрдүүлдэг. Иймд энэхүү загвар нь монгол хэл дээрх асуултыг англи хэл дээрх баримт мэдээлэлтэй утгын түвшинд харьцуулж, олон хэл дамнасан хайлтын үр дүнг сайжруулах давуу талтай юм.

3.4 Transformer архитектур ба Attention механизм

3.4.1 Attention механизмын үндэс

Transformer архитектур нь орчин үеийн ихэнх том хэлний загваруудын (LLM) үндсэн суурь болдог. Энэхүү архитектурын гол ойлголт болох self-attention механизм нь өгүүлбэр доторх үг бүрийг бусад бүх үгтэй нь нэгэн зэрэг харьцуулж, аль үг ямар үгтэй илүү утгын холбоотой болохыг тодорхойлдог арга юм.

Self-attention-ийг дараах байдлаар ойлгож болно: өгүүлбэрийн үг бүр өөрийгөө “асуулт (Query)”, бусад үгсийг “түлхүүр (Key)” болон “утга (Value)” хэлбэрээр төлөөлөн авч үздэг. Ингэснээр тухайн үг нь өөрт хамгийн их хамааралтай бусад үгсийг олж, тэдгээрт илүү их анхаарал хандуулдаг.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Энд Q (Query), K (Key), V (Value) матриц нь оролтын векторуудаас тооцоологдох бөгөөд d_k нь Key вектор хэмжээ юм.

Математикийн хувьд энэ үйл явц нь үг хоорондын хамаарлыг тооцоолж жинлэх хэлбэрээр илэрхийлэгдэнэ. Өөрөөр хэлбэл үг бүр бусад үгтэйгээ хэр зэрэг холбоотойг оноогоор үнэлж, тэр оноог нормчлон (softmax) магадлалын жин болгодог. Дараа нь эдгээр жинг ашиглан тухайн өгүүлбэрийн мэдээллийг хамгийн чухал хэсгүүдэд төвлөрүүлж нэгтгэдэг. Ингэснээр модель өгүүлбэрийн утгыг зөвхөн дараалалд тулгуурлахгүйгээр, бүх үгсийн хоорондын уялдаа холбоонд үндэслэн илүү нарийвчлалтай ойлгох боломжтой болдог.

3.4.2 LLM-ийн бичвэр үүсгэх чадвар

Том хэлний загварууд нь их хэмжээний бичвэр дээр сургагдсанаар хүний хэлийг ойлгож, хариулт үүсгэх чадвартай болдог. Гэхдээ эдгээр загварт хоёр гол сул тал байдаг:

- **Knowledge cutoff** — Сургалтын өгөгдлийн хугацааны хязгаараас шинэ мэдээллийг мэдэхгүй.
- **Hallucination** — Мэдэхгүй зүйлд буруу, гэхдээ итгэлтэй харагдах хариулт үүсгэх хандлага.

RAG технологи нь яг эдгээр сул талыг нөхдөгөөрөө онцлог юм: загварын сургагдсан мэдлэг төдийгүй шинэ баримт бичгийн агуулгыг ашигладаг.

3.5 ONNX ба загварын оновчлолын онол

3.5.1 ONNX стандарт

ONNX нь нейрон сүлжээний загваруудыг хэрэглэгдэхүүнээс үл хамааран нэгэн стандарт форматад хөрвүүлэхэд зориулагдсан нээлттэй формат юм. Энэхүү стандартын тусламжтайгаар PyTorch, TensorFlow, Keras зэрэг өөр өөр framework дээр хөгжүүлсэн загваруудыг нэгтгэн ONNX формат руу хөрвүүлж, улмаар өөр орчинд дахин хөгжүүлэлт хийх шаардлагагүйгээр шууд ашиглах боломжтой болдог. Ингэснээр загварын хоорондын шилжилт болон тооцооллын явцыг хялбарчилж, хэрэгжүүлж буй орчноос хамаарах хязгаарлалтыг бууруулдаг.

3.5.2 INT8 Quantization

Quantization нь загварын жинг бага нарийвчлалтай тоон хэлбэрт шилжүүлж хэмжээ болон хурдыг сайжруулах арга юм:

- **FP32** — Стандарт 32-bit float. Нарийвчлал өндөр, хурд удаан.
- **FP16** — 16-bit float. GPU-д тохиромжтой, FP32-аас ~ 2 дахин хурдан.
- **INT8** — 8-bit integer. CPU-д хамгийн тохиромжтой, FP32-аас ~ 4 дахин хурдан, нарийвчлалын алдагдал 1% хүрэхгүй.

ONNX Runtime-д ашиглагддаг квантчилал нь нейрон сүлжээний загварын тоон утгуудын нарийвчлалыг багасгаж, загварын хэмжээг багасгах болон ажиллах хурдыг нэмэгдүүлэх арга юм. Жишээлбэл, ихэнх загварууд жингүүдээ 32-битийн бутархай тоогоор (FP32) хадгалдаг бол квантчиллын үед эдгээрийг 8-битийн бүхэл тоо (INT8) болгон хөрвүүлдэг. Ингэснээр санах ойд эзлэх хэмжээ багасч, тооцоолол илүү хурдан гүйцэтгэгдэх боломж бүрддэг.

3.5.3 Intel OpenVINO хурдасгал

Intel OpenVINO нь Intel процессорын боломжуудыг ашиглан нейрон сүлжээний загварын ажиллах хурдыг нэмэгдүүлэх зориулалттай хэрэгсэл юм. Энэ нь процессорт байдаг тусгай зааврууд (жишээлбэл, AVX-512, AMX)-ыг ашигласнаар загварын тооцооллыг илүү хурдан гүйцэтгэх боломжийг олгодог.

ONNX Runtime-д OpenVINOExecutionProvider-ийг ашигласнаар нэмэлт код бичихгүйгээр загварын гүйцэтгэлийг мэдэгдэхүйц сайжруулах боломжтой. Туршилтын үр дүнгээс харахад CPU дээр боловсруулах хурд 7 chunk/с байсан бол OpenVINO ашигласнаар 63 chunk/с болж, ойролцоогоор 9 дахин хурд нэмэгдсэн байна.

3.6 Мэдээлэл хайлтын үнэлгээний үзүүлэлтүүд

3.6.1 Precision@K ба Recall@K

Хайлтын системийн чанарыг үнэлэхэд хэрэглэгддэг үндсэн үзүүлэлтүүд нь нарийвчлал (Precision) болон санамж (Recall) юм. Эдгээр нь систем хэрэглэгчийн асуултад хэр зөв болон хэр бүрэн хариулж байгааг хэмждэг.

$$\text{Precision@K} = \frac{\text{Хамааралтай олдсон баримт}}{K}$$

Precision@K (нарийвчлал) нь хайлтын системээс буцаасан эхний K баримтын дотор хэд нь тухайн асуулттай үнэхээр хамааралтай байгааг илэрхийлнэ. Өөрөөр хэлбэл, системийн гаргасан үр дүнгийн зөв байдлын түвшинг хэмждэг үзүүлэлт юм. Жишээлбэл, K=5 үед буцаасан 5 үр дүнгээс 4 нь зөв байвал Precision@5 = 0.8 болно.

$$\text{Recall@K} = \frac{\text{Хамааралтай олдсон баримт}}{\text{Нийт хамааралтай баримт}}$$

Энд K нь хайлтаас буцаасан баримтын тоо. Одоогийн систем жижиг учир нь Top-K параметрийг 1–10 хооронд тохируулах ба утгыг 5 тохиромжтой гэж үзсэн.

3.6.2 Confidence Score

Энэ системд confidence score-ийг `calculate_confidence_score()` функцээр суурь retrieval оноон дээр нэмэлт шалгууруудын жинг нэмж тооцоолдог. Аргазүйн хэсэгт уг тооцооллын хүснэгтэн задаргааг Хүснэгт 6-д нэгтгэн үзүүлэв.

3.7 Ижил төрлийн судалгааны ажлуудын харьцуулалт

Хүснэгт 2: Ижил төрлийн системүүдийн харьцуулалт

Систем	Офлайн	Hybrid	Формат	Монгол	Онцлог
Lewis et al. RAG	Үгүй	Үгүй	Вики	Үгүй	Анхны RAG судалгаа
LangChain RAG	Үгүй	Зарим	PDF,TXT	Үгүй	API шаарддаг
PrivateGPT	Тийм	Үгүй	PDF,TXT	Үгүй	Зөвхөн 2 формат
Haystack	Тийм	Тийм	Олон	Үгүй	Серверт суурилсан
RAG (Анар)	Тийм	Тийм	10+	Тийм	ONNX+OpenVINO

1. **Lewis et al. (2020)** — RAG-ийн анхны судалгаа. DPR + BART загвараар Wikipedia-аас хариулт үүсгэсэн [1]. Гэвч локал файлд хэрэглэх боломжгүй, офлайн горим дэмжихгүй.
2. **LangChain RetrievalQA** — OpenAI API шаарддаг, офлайн орчинд нэмэлт тохиргоо хэрэгтэй [5]. Хамгийн өргөн хэрэглэгддэг боловч өгөгдлийн нууцлалын асуудалтай.
3. **PrivateGPT** — Бүрэн локал ажилладаг ч зөвхөн PDF, TXT дэмждэг, Hybrid хайлт хэрэгжүүлээгүй [12]. ONNX оновчлол байхгүй тул CPU дээр удаан.
4. **Haystack (deepset)** — Серверт суурилсан архитектур тул хэрэглэгчийн компьютерт хүнд [11]. Pipeline тохиргоо нарийн мэдлэг шаарддаг.
5. **Benedetti (2024/2025)** — RAGAS framework-ээр RAG системийг үнэлсэн судалгаа [8]. API-based тул офлайн ашиглах боломжгүй.

Дээрх харьцуулалтаас харахад ихэнх RAG систем нь нэг хайлтын арга, онлайн API, эсвэл цөөн файлын формат дэмждэг. Энэ систем нь **Hybrid хайлт (3 арга)**, **бүрэн сүлжээгүй орчинд**, **10 файлын формат унших**, **ONNX оновчлол буюу бага тооцоолол** хослуулсанаараа ялгаатай.

4 Судалгааны арга зүй

Энэхүү судалгаа нь хэрэглээний чиглэлийн судалгаа бөгөөд хүмүүсийн хувийн болон компанийн файл, бичиг баримттай харьцах, хайх хугацааг багасган илүү үр дүнтэй ажиллахад чиглэсэн. Чанарын судалгаа ашиглан ямар хиймэл оюун ухааны модел илүү хариулт гаргаж байгааг үнэлсэн.

4.1 Системийн ерөнхий архитектур

Систем нь гурван гол давхаргаас бүрдэнэ: оролтын давхарга (файл унших, chunk болгох), индексийн давхарга (FAISS + BM25 + кэш), хариултын давхарга (Hybrid хайлт + LLM + UI). Дараах диаграмм нь системийн өгөгдлийн урсгалыг харуулна.



Зураг 2: RAG системийн өгөгдлийн урсгалын диаграмм (DFD): индексжүүлэх үе (файл унших, chunking, embedding, FAISS/BM25) ба хайлтын үе (query боловсруулах, fusion, confidence/rerank, LLM, UI)

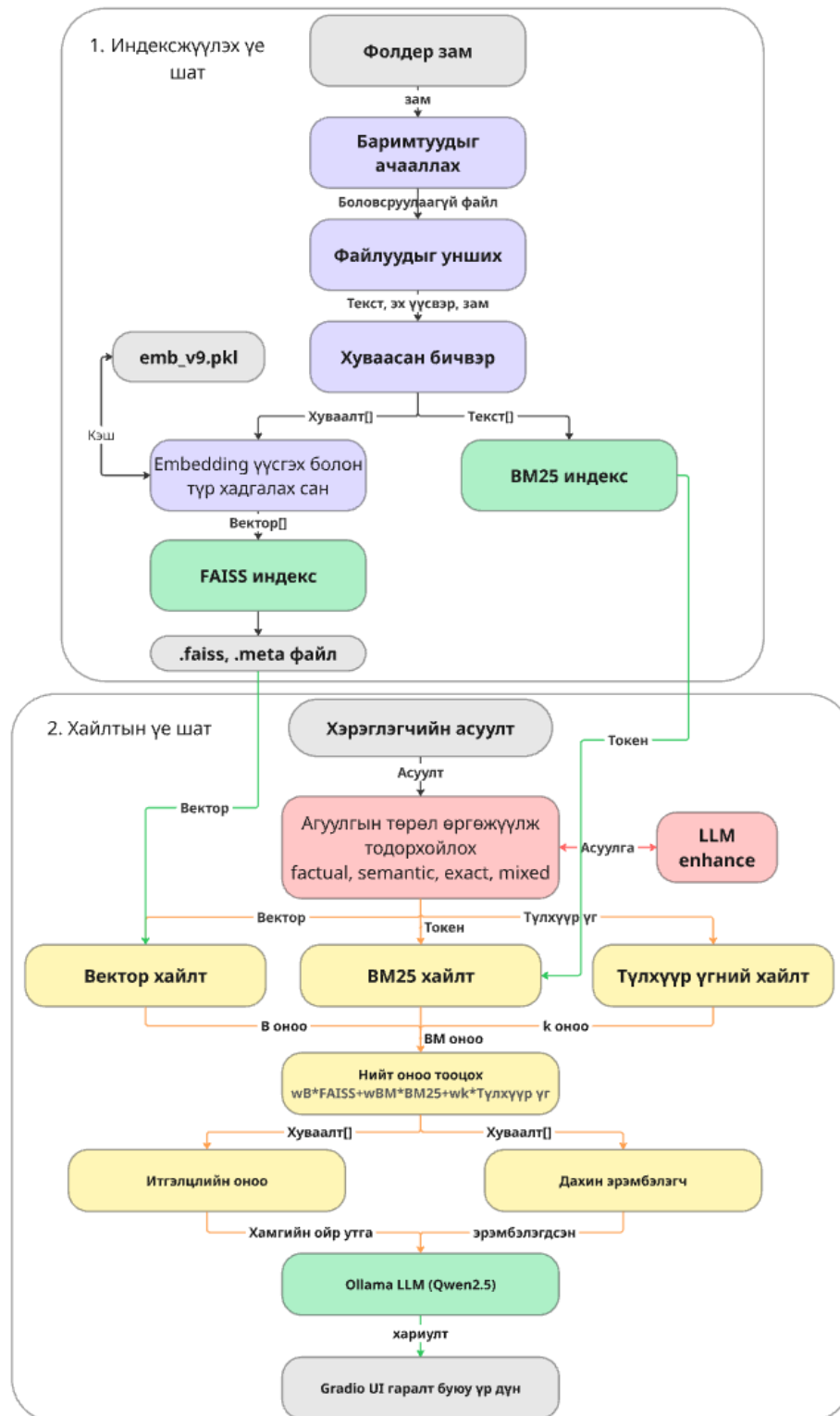
Зураг 2-т үзүүлсэн урсгал нь App.py-ийн бодит хэрэгжилттэй нийцнэ. Индексжүүлэх үед `load_documents()` нь файлыг ачаалахаас өмнө хэмжээг (50MB босо) шалгаж, дэмжигдэх өргөтгөлүүдийг шүүдэг. Дараа нь `chunk_text()` нь `CHUNK_SIZE=600`, `CHUNK_OVERLAP=80` тохиргоогоор хэсэгчлэж, `EmbeddingCache(emb_v9.pkl)`-аар MD5 кэш ашиглан давтагдсан embedding-ийг дахин тооцоолохоос сэргийлнэ. Embedding үүсгэхдээ боломжтой үед ONNX/OpenVINO ашиглаж, боломжгүй үед `SentenceTransformer fallback`-аар ажиллан, эцэст нь chunk-ийн тооноос хамааран FAISS `IndexFlatIP` эсвэл `IndexIVFFlat` болон BM25 индексүүдийг зэрэг хадгалдаг.

Кодын хэрэгжилтийн хариуцлагын түвшинд энэхүү урсгалыг таван модульд товчлон харж болно: M1 (оролт, файл уншилт, шүүлт), M2 (chunk, embedding, индекс), M3 (hybrid retrieval ба confidence), M4 (LLM хариулт үүсгэх), M5 (UI callback ба индексийн lifecycle). Энэ задаргаа нь Зураг 2-ийн индексжүүлэх ба хайлтын хоёр үе шаттай логиктой нийцдэг.

4.2 Системийн ажиллах дараалал

Систем нь хоёр үндсэн үе шатаар ажилладаг: **(1) Индексжүүлэх үе** — фолдерын файлуудыг уншиж, chunk болгон (хувааж), embedding + FAISS/BM25 индекс үүсгэж хадгална. **(2) Хайлтын үе** — хэрэг-

лэгч асуулт оруулахад холимог хайлт хийж, Тор-К chunk (хамгийн ойр гэж үзсэн хуваалт)-ийг LLM-д дамжуулан хариулт үүсгэнэ.



Зураг 3: Системийн ажиллах дарааллын ерөнхий зураглал

Зураг 3-ийн дагуу ажиллах дарааллыг товчлон үзүүлбэл:

- Индексжүүлэх үе:** `load_documents()` → `chunk_text()` → `BM25Index.build(all_texts)` → `embedding` тооцоолол → `FAISS индекс` → `save()`.
- Хайлтын үе:** `query` ангилал + `expansion` → `vector/BM25/keyword candidate` → `score fusion` →

(нөхцөлтэй) reranker → Top-K context → LLM streaming хариулт → UI гаралт.

Системийн боловсруулалтын урсгал суурь түвшинд индексжүүлэх үеэс эхэлдэг: энэ алхамыг нэг удаа гүйцэтгэх бөгөөд embedding кэш ашигласнаар зөвхөн өөрчлөгдсөн файлуудыг дахин боловсруулж, тооцооллын зардлыг бууруулна. Үүний дараа хайлтын шатанд асуулгыг `classify_query_type()` болон `expand_query()` функцээр шинжилж, *factual* (баримт), *semantic* (утгын), *exact* (шууд таарах), *mixed* (холимог) гэсэн дөрвөн горимын аль жинлэлтийг хэрэглэхийг нөхцөлд тулгуурлан `WEIGHT_PROFILES`-оос сонгодог (дэлгэрэнгүйг §4.9.1-д үзнэ үү).

Илүү ахисан түвшинд Ollama загвар сонгогдсон үед `_ollama_ping()` функцээр Ollama LLM ажиллаж байгаа эсэхийг эхэлж шалгана; шалгалт амжилттай бол `combined_llm_enhance()` нь анхны query-г rewritten query болон keywords болгон өргөтгөж, BM25/FAISS-аар нэмэлт candidate таталтын чанар ба хамрах хүрээг сайжруулахад ашиглагдана. Харин `hyde_generate()` болон `self_reflect_search()` нь кодоод буй нэмэлт helper боломжууд бөгөөд үндсэн `answer_stream()` урсгалд заавал ажиллах алхам биш юм. Мөн confidence score нь `CONFIDENCE_THRESHOLD` босготойгоор үр дүнг тайлбарлах дохио хэлбэрээр UI-д харуулагддаг.

4.3 Файл уншигч модуль

Файл уншигч модуль нь `_read_one()` функцаар хэрэгждэг бөгөөд уншилтын урсгалын төв цэг болдог. Энэхүү функц нь хоёр үндсэн үүрэгтэй: файлын төрөлд тохирох уншигчийг автоматаар сонгох, мөн уншилтын үеийн алдааг тогтвортой байдлаар боловсруулах. Уншилт эхлэхээс өмнө урьдчилсан шалгалт хийж, файлын хэмжээг 50MB босготой харьцуулан хэт том файлыг алгасахын зэрэгцээ хоосон файлыг уншилтанд оруулахгүй.

4.3.1 Дэмжигдэх форматууд

Модуль дэмждэг файлын төрлүүд болон тэдгээрт ашиглагдах санг Хүснэгт 3-д нэгтгэн үзүүлэв.

Хүснэгт 3: Дэмжигдэх файлын төрлүүд ба уншигч

Өргөтгөл	Тайлбар	Ашигласан сан
.pdf	Баримт бичиг	PyMuPDF / pdfplumber / pypdf (4 шатлал)
.docx	Word баримт	python-docx
.txt, .md, .log, .rst	Текст файлууд	Built-in (4 encoding дэмжинэ)
.csv, .tsv	Хүснэгт	pandas
.xlsx, .xls	Excel	pandas (openpyxl)

Энэхүү хамрах хүрээ нь баримт бичиг, хүснэгтэн өгөгдөл, энгийн текстийн нийтлэг хэрэглээг бүрэн дэмжих зорилготой бөгөөд дараах дэд хэсгүүдэд тайлбарлах шүүлт, нөөц шилжилтийн механизмуудтай уялдан ажилладаг.

4.3.2 Encoding автоматаар илрүүлэх

Текст файл унших үед `read_txt()` функц файл зөв уншигдаж байгаа эсэхийг шалгахын тулд UTF-8, UTF-16, CP1251, Latin-1 гэсэн хэд хэдэн хэлбэрээр дарааллаар туршиж үздэг. Аль хэлбэрээр файл алдаагүй уншигдвал түүнийг ашиглан текстийг авна. Ингэснээр Монгол кирилл агуулгыг CP1251 хэлбэрээр хадгалсан хуучин файлуудыг ч алдаагүй унших боломжтой болдог.

4.3.3 PDF-ийн 4 шатлалт нөөц шилжилтийн механизм

PDF файлаас текст гарган авах нь бүтэц, фонт, кодчиллын ялгаанаас шалтгаалан хамгийн эмзэг үе шат тул `read_pdf()` функцийг дараах 4 шатлалт нөөц шилжилтийн механизмаар хэрэгжүүлсэн:

1. **PyMuPDF — Шатлал 1 (хамгийн хурдан):** `fitz.open()` функцаар PDF-ийг нээж, хуудас бүрээс `get_text("text")` аргаар текст гаргана. Хэрэв гарсан текст хоосон эсвэл хэт богино байвал `get_text("dict")` аргыг ашиглан баримт бичгийн жижиг хэсгүүдийн (`span`) текстийг тус тусад нь цуглуулж авна. Ингэснээр баримтын доторх үг, өгүүлбэрийн хэсгүүдийг алдалгүй унших боломжтой болдог. Харин цуглуулсан текст 10-аас дээш тэмдэгттэй байвал түүнийг амжилттай уншигдсан гэж үзээд шууд буцаана.
2. **pdfplumber — Шатлал 2 (CJK/Японы PDF):** Япон, хятад фонттой PDF дээр PyMuPDF хангалттай текст гаргаж чадахгүй тохиолдолд `pdfplumber.open()` функцийг ашиглан `pg.extract_text()` аргаар хуудас бүрийг уншина.
3. **pypdf — Шатлал 3 (ерөнхий нийцтэй):** Эхлээд файлын эхний 4 байтыг `b"%PDF"` тэмдэгт мөртэй тохирч байгаа эсэхээр PDF формат мөн эсэхийг тодорхойлно. Нөхцөл хангагдвал `PdfReader(strict=False)` ашиглан хуудас тус бүрийн текстийг уншина. Хэрэв файл нь `scan` хийсэн зурган PDF бол “бичвэр гарахгүй” гэдгийг командын хэсэгт мэдэгдэнэ.
4. **Placeholder — Шатлал 4 (файлыг хэзээ ч чимээгүй алдахгүй):** Дээрх гурван шат амжилтгүй болсон тохиолдолд PyMuPDF номын санг ашиглан PDF файлын хуудасны тоог тодорхойлж, [Зурган PDF] нэр | N хуудас хэлбэрийн түр орлуулагч текст үүсгэнэ. Ингэснээр тухайн файл нь индексэд бүртгэгдэж, хайлтын үр дүнд харагдах боломжтой болох ба хэрэглэгчдэд уг файл нь зөвхөн зурган хэлбэрийн PDF гэдгийг мэдээллэнэ.

4.3.4 Шүүлтийн механизм

Файлуудыг уншиж индексжүүлэхээс өмнө систем нь өгөгдлийн чанарыг хангах болон боловсруулалтын гүйцэтгэлийг сайжруулах зорилгоор хэд хэдэн шаталсан шүүлтийг хэрэгжүүлдэг.

- **Директорийн шүүлт** — Хувилбарын удирдлагын сан, виртуал орчин, кэш болон тестийн директор зэрэг техникийн зориулалттай хавтаснуудыг автоматаар алгасна. Шалгалтыг тухайн файлын байрлаж буй замын бүх дээд директорийг хамруулан гүйцэтгэнэ.
- **Файлын төрлийн шүүлт** — Програмын эх код, хоёртын файл болон медиа төрлийн файлуудыг индексжүүлэх үйл явцаас хасна. Ингэснээр текстэн агуулгатай баримт бичгүүдийг боловсруулахад төвлөрөх боломж бүрддэг.
- **Тусгай файлын шүүлт** — Системийн түр файлууд, `template` бүтэцтэй файл болон тестийн өгөгдөлд хамаарах файлуудыг автоматаар ялган алгасна.
- **Хэмжээ болон агуулгын шүүлт** — Хэт том хэмжээтэй файлууд, хоосон файл, эсвэл маш бага хэмжээний текст агуулсан файлуудыг индексжүүлэхээс өмнө шүүнэ.
- **Параллель боловсруулалт** — Олон файлыг зэрэг унших боломжтой олон урсгалт боловсруулалтыг ашигласнаар нийт индексжүүлэх үйл явцын хурдыг нэмэгдүүлдэг.

4.4 Chunking (хэсэгчлэл)

`chunk_text()` функц нь баримт бичгийн текстийг тогтмол урттай жижиг хэсгүүдэд хуваах үүрэгтэй. Кодын тохиргоонд нэг хэсгийн хэмжээг `CHUNK_SIZE = 600`, тэмдэгт гэж, харин дараалсан хоёр хэсгийн хооронд давхцах хэмжээг `CHUNK_OVERLAP = 80` тэмдэгт гэж тодорхойлсон. Өөрөөр хэлбэл, шинэ хэсэг үүсэх үед өмнөх хэсгийн сүүлийн 80 тэмдэгт давхар орж ирдэг бөгөөд ингэснээр текстийн утга, өгүүлбэрийн уялдаа алдагдахгүй хадгалагддаг.

4.4.1 Хуваах алгоритм

1. Бичвэрийн олон давхар зайг `re.sub(r"\s+", " ", text)` илэрхийллээр нэг зай болгон цэвэрлэнэ.
2. Бичвэрийн урт хуваалтын хэмжээнд хүрмэгц хуваалтыг эхлүүлнэ.
3. Хуваах цэгийг хэсгийг тасалдалтын дарааллаар `". ", ".\n", "!\n", "?\n", "\n\n", "\n", "; ", ", ", " "` гэж шалгана.
4. `Overlap` хэсгийг дахин оруулахдаа `start = end - overlap` томъёог ашиглана.
5. `MIN_CHUNK_LEN = 10` нөхцөлийн дагуу 10 тэмдэгтээс богино chunk-ийг хасна.

4.4.2 Chunk мета өгөгдөл

Текстийг хэсгүүдэд хуваасны дараа chunk бүртэй холбоотой нэмэлт мэдээллийг мета өгөгдөл хэлбэрээр хадгална. Үүнд дараах мэдээллүүд багтана.

- `text` — chunk-ийн бичвэрийн агуулга
- `source` — эх файлын нэр (Unicode NFC normalize хийгдсэн)
- `path` — файлын бүтэн зам

4.4.3 Параллель chunking

`build_chunks_parallel()` функц нь `ThreadPoolExecutor`-ийг ашиглан олон файлыг зэрэгцүүлэн chunk болгон хуваадаг. Файл бүрийн chunk-уудыг `file_chunks`-д `{path: [chunks]}` бүтэцтэйгээр хадгална.

4.5 Embedding ба ONNX оновчлол

4.5.1 EmbedEngine класс

`EmbedEngine` класс нь `lazy loading` зарчмаар ажилладаг. Өөрөөр хэлбэл, `embedding` үүсгэх `encode()` функц анх дуудагдах үед л загварыг санах ойд ачаална.

`Embedding` тооцоолох загварыг систем дараах дарааллаар сонгоно.

1. Хэрэв CPU горимд ажиллаж байгаа бөгөөд ONNX загвар байрлах хавтас илэрвэл `ORTModelForFeature` ашиглан `embedding` үүсгэнэ.
2. Хэрэв OpenVINO боломжтой бол Intel процессорын хурдасгалыг ашиглан тооцооллыг хурдан гүйцэтгэнэ.
3. Хэрэв OpenVINO ашиглах боломжгүй бол `model_q8.onnx` (INT8 квантчилсан загвар) эсвэл `model.onnx` (FP32 загвар)-ыг ашиглана.

4. Хэрэв ONNX загвар байхгүй бол SentenceTransformer номын сангийн PyTorch загварыг ашиглан embedding үүсгэнэ.
5. Мөн GPU (CUDA эсвэл MPS) боломжтой бол тооцооллыг автоматаар GPU дээр гүйцэтгэнэ.

Ингэснээр систем нь тухайн төхөөрөмжийн боломжит хамгийн тохиромжтой загвар болон тооцооллын орчныг автоматаар сонгож, текстийг вектор хэлбэрт хөрвүүлэх (embedding үүсгэх) ажиллагааг гүйцэтгэдэг.

4.5.2 Загварын сонголт

Хүснэгт 4: Embedding загварууд

Түлхүүр нэр	HuggingFace загвар	Хэмжээ	Монгол	Урт
Олон хэлний хурдан горим	paraphrase-multilingual-MiniLM-L12-v2	470MB	Тийм	384
Олон хэлний өндөр чанарын горим	intfloat/multilingual-e5-small	470MB	Тийм	384
Маш хурдан горим	paraphrase-MiniLM-L3-v2	17MB	Үгүй	384
Хурдан горим	all-MiniLM-L6-v2	80MB	Үгүй	384

Өгөгдмөлөөр **Олон хэлний хурдан горим** (multilingual-fast) загварыг ашигладаг. **Олон хэлний өндөр чанарын горим** (multilingual-e5-small) загварыг сонгох үед use_prefix=True тохиргоог идэвхжүүлж, асуулгад "query: ", баримтад "passage: " гэсэн угтвар нэмж embedding үүсгэнэ. Ингэснээр загвар тухайн текст асуулга эсвэл баримт болохыг зөв ялгаж ойлгож, асуулга болон холбогдох баримтын утгын хамаарлыг илүү оновчтой тодорхойлсноор хайлтын нарийвчлалыг сайжруулдаг.

4.5.3 ONNX INT8 quantization

ONNX нь нейрон сүлжээний загварыг янз бүрийн програм, тооцооллын орчинд ажиллуулах боломж олгодог загварын стандарт формат юм. Загварыг ONNX формат руу хөрвүүлсний дараа INT8 quantization хийснээр жингүүдийг бага нарийвчлалтай (8 бит) хэлбэрт хувиргаж, загварын хэмжээг багасгана. Үүний үр дүнд загварын хэмжээ ойролцоогоор 50

Мөн Intel процессорт зориулсан OpenVINO provider ашиглах боломжтой үед ONNX загварыг Intel-ийн тусгай хурдасгал ашиглан ажиллуулдаг тул гүйцэтгэлийн хурд дахин 3–5 дахин нэмэгдэх боломжтой. Quantization хийсний дараа үүссэн ONNX загварыг model_q8.onnx нэртэй файлаар хадгалдаг.

4.5.4 GPU-д тулгуурласан batch хэмжээний оновчлол

Систем нь embedding тооцоолох үед batch хэмжээ (нэг дор боловсруулах текстийн тоо)-г төхөөрөмжийн санах ойн хэмжээнд үндэслэн автоматаар тохируулдаг.

_gpu_batch() функц нь GPU-ийн VRAM хэмжээг шалгаж batch хэмжээг нэмэгдүүлнэ. Жишээлбэл, 16GB ба түүнээс их VRAM-тай бол batch хэмжээг 8 дахин, 8GB үед 4 дахин, 4GB үед 2 дахин нэмэгдүүлнэ. Харин GPU ашиглахгүй, зөвхөн CPU дээр ажиллаж байгаа тохиолдолд batch хэмжээг өөрчлөхгүй.

Мөн системийн RAM хэмжээг харгалзан _opt_batch() функц тохиромжтой batch хэмжээг сонгодог. Үүнд 8GB хүртэл RAM-тай үед 8, 16GB үед 16, 32GB үед 32, түүнээс их RAM-тай бол 64 хэмжээтэй batch ашиглана.

Ингэснээр систем нь төхөөрөмжийн нөөцийг үр ашигтай ашиглаж, embedding тооцооллыг илүү хурдан гүйцэтгэх боломжтой болдог.

4.6 Embedding кэш систем

EmbeddingCache анги нь embedding-ийг олон дахин тооцоолохоос сэргийлэх зорилгоор MD5 дээр суурилсан хоёр түвшний кэшийн механизм ашигладаг. Өөрөөр хэлбэл өмнө нь боловсруулсан бичвэрийн embedding-ийг хадгалж, дараагийн удаа шууд ашиглах боломжийг олгодог.

Кэшийн ажиллах зарчим

1. Текстын хэсэг (chunk) бүрээс MD5-д суурилсан давтагдашгүй таних утга үүсгэнэ. Энэ нь тухайн текстыг өвөрмөц байдлаар танихад ашиглагдана.
2. Програм эхлэх үед өмнө хадгалсан embedding-үүдийг `emb_v9.pkl` файлаас санах ойд ачаална.
3. Encoding буюу кодыг тайлж унших үед кэшийг шалгаж, өмнө нь тооцоологдсон бичвэр байвал embedding-ийг шууд ашиглана. Ингэснээр давхардсан бичвэрийг дахин тооцоолох шаардлаггүй болно.
4. Шинээр үүссэн embedding-үүдийг кэшд нэмээд, үндсэн процессыг саатуулахгүйгээр тусдаа процессын урсгал ашиглан хадгална.
5. Кэш файлыг хадгалахдаа эхлээд `.tmp` түр файлд бичиж, дараа нь солих зарчмаар хадгалж явна.

Ингэснээр embedding тооцооллыг дахин давтахгүй, системийн хурд болон үр ашгийг сайжруулдаг.

4.7 FAISS вектор индекс

`HybridStore.build()` автоматаар сонгодог. Ингэснээр жижиг өгөгдөл дээр өндөр нарийвчлалтай хайлт хийх, харин том өгөгдөл дээр хурдан хайлт хийх хоёрын тэнцвэрийг хангана.

- $\text{Chunk} \leq 5,000$ үед **IndexFlatIP** — индексийг ашиглана. Энэ нь бүх векторуудыг шууд харьцуулж хайлт хийдэг тул нарийвчлал өндөр боловч том өгөгдөл дээр удаан болно.
- $\text{Chunk} > 5,000$ үед **IndexIVFFlat** — индексийг ашиглана. Энэ үед систем эхлээд векторуудыг төстэй байдлаар нь бүлэглэн хэсгүүдэд хуваадаг (өөрөөр хэлбэл ижил төрлийн мэдээллүүдийг нэг ангилалд оруулна). Дараа нь хайлтын үед бүх өгөгдлийг шалгахын оронд зөвхөн тухайн асуулгад хамгийн ойр хэсгүүдийг л шалгаж, илүү хурдан хайлт хийдэг.

Индексийг `.faiss` өргөтгөлтэй файлд хадгална. Энэ файл нь embedding векторуудын хайлтын бүтэц буюу үндсэн индексийг агуулдаг. Харин мета өгөгдөл болох chunk-ийн жагсаалт, ашигласан embedding загварын нэр, хадгалсан огноо зэрэг нэмэлт мэдээллийг `.meta` өргөтгөлтэй `pickle` файлд тусад нь хадгална. Ингэснээр хайлтын индекс болон тайлбар мэдээллийг хооронд нь ялгаж, системийг илүү зохион байгуулалттай, удирдахад хялбар болгодог.

4.8 BM25 статистик индекс

Түлхүүр үгт суурилсан хайлтыг хэрэгжүүлэхдээ систем нь BM25 алгоритмыг ашигладаг. Энэхүү алгоритм нь баримт бичгүүдийн дундаас хэрэглэгчийн асуулгад хамгийн их хамааралтай хэсгүүдийг түлхүүр үгийн давтамж болон тархалтад үндэслэн эрэмбэлж олдог. Хэрэв уг алгоритмыг хэрэгжүүлэх шаардлагатай програмын сан системд суусан байхгүй тохиолдолд орлуулах механизм (fallback)

идэвхжиж, TF-IDF дээр суурилсан хайлтын аргыг ашиглана [TF-IDF тайлбар: §3.3.1, х. 5]. Ингэснээр гадны сан дутуу байсан ч системийн үндсэн хайлтын ажиллагаа тасалдахгүй үргэлжлэх боломж бүрддэг.

4.8.1 Токенчлох арга

`_tokenize()` функц нь текстийг хэлний онцлогоос хамааран хэсэглэж, латин болон кирилл үгс, мөн CJK (Хятад, Япон, Солонгос) тэмдэгтүүдийг тус тусад нь ялган боловсруулдаг.

- `re.findall(r'\w+', txt)` — үсэг болон тоон тэмдэгтүүдээс бүрдэх үгсийг ялган авна (латин болон кирилл үгсийг хамарна).
- CJK тэмдэгтэд суурилсан текстийн хайлтын дүрэм — Хятад, Япон, Солонгос хэлний тэмдэгтүүдийг тусад нь ялгана.

4.8.2 Хайлтын боловсруулалт

Нэг хайлтын асуулгаас гадна `_build_bm25_queries()` функц нь асуулгын өргөтгөл (query expansion) болон Ollama загварын мэдлэгээс гаргасан түлхүүр үгсийг ашиглан олон төрлийн BM25 хайлтын чанарыг дэлгэрүүлэх үгнүүд үүсгэдэг. Ингэснээр нэг асуулгыг зөвхөн ганц хэлбэрээр биш, өөр өөр утгын хувилбаруудаар хайж илүү өргөн хүрээтэй үр дүн гаргах боломжтой болдог. Гарсан үр дүнг нэгтгэхдээ нэг индексийн оноо хэд хэдэн удаа давтагдан ирвэл хамгийн өндөр оноог хадгалах зарчмаар ажилладаг бөгөөд энэ нь тухайн баримтын хамгийн сайн тохирох байдлыг илэрхийлнэ.

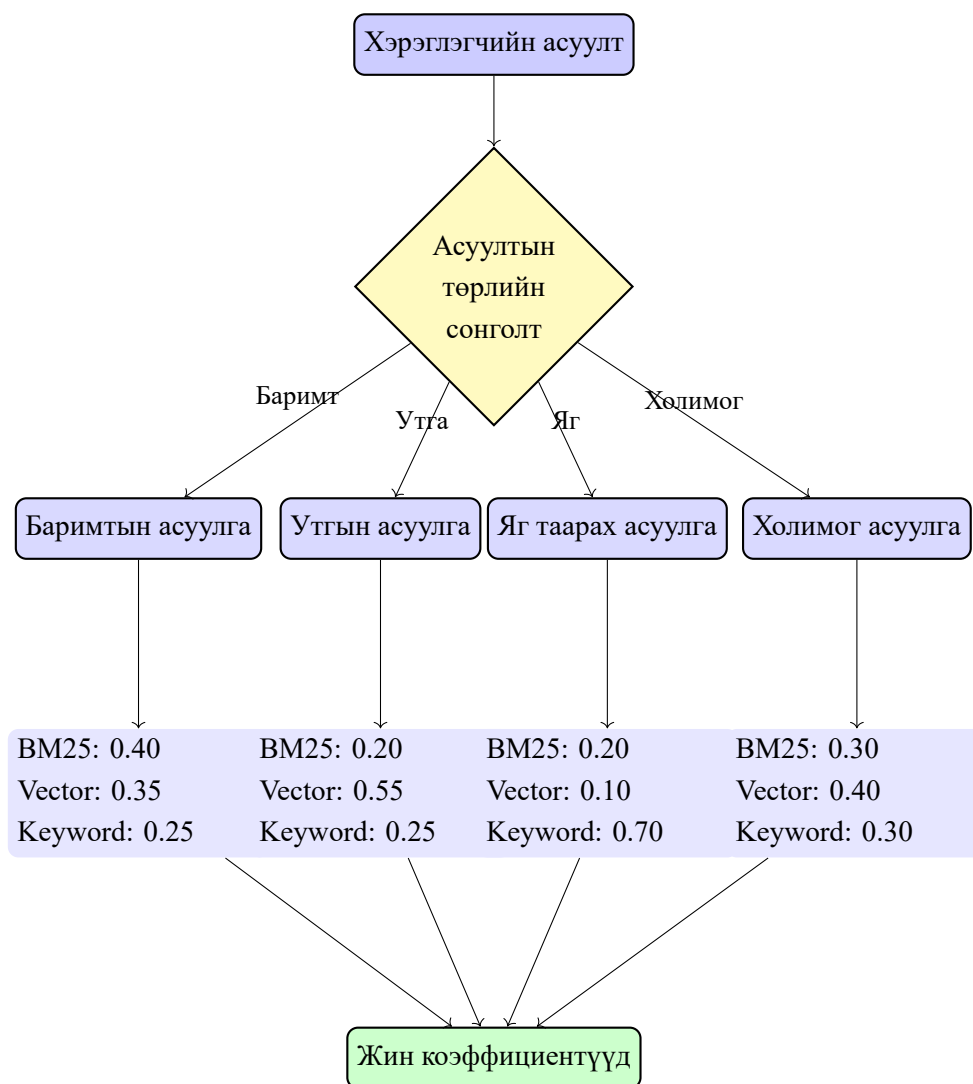
4.9 Холимог хайлтын хэрэгжүүлэлт

4.9.1 Дөрвөн горимын жинлэлт

Хайлтын асуултын шинж чанарт нийцүүлэн дараах дөрвөн жинлэлтийн профайлыг тодорхойлсон:

Хүснэгт 5: Хайлтын горим бүрийн жинлэлт

Горим	BM25	Vector	Keyword
factual	0.40	0.35	0.25
semantic	0.20	0.55	0.25
exact	0.20	0.10	0.70
mixed	0.30	0.40	0.30



Зураг 4: Асуултын ангилал ба жин сонголтын урсгал

`classify_query_type()` функц нь асуултыг шинжилж горимыг сонгоно: ишлэл болон яг гэх үг илэрвэл *exact*; тоон утга, хэдэн/хэзээ/хэн зэрэг асуулгын үг илэрвэл *factual*; яагаад/тайлбарла зэрэг семантик түлхүүр илэрвэл *semantic*; бусад тохиолдлыг *mixed* гэж ангилна.

4.9.2 Нэгтгэлийн дараалал

`answer_stream()` функц нь хоёр өөр хайлтын урсгал дээр үндэслэн ажилладаг. Хэрэв Ollama загвар ашиглагдаж байвал сайжруулсан хайлтын урсгал болох `enhanced_search_pipeline()`-ийг ажиллуулна. Харин Ollama ашиглагдаагүй тохиолдолд систем үндсэн хайлтын аргаар `store.search()` функцийг ашиглан нөөц арга байдлаар үр дүн гаргана. Ингэснээр систем нь загварын тохиргооноос хамаарахгүйгээр үргэлж ажиллах боломжтой болж, тасралтгүй хариу өгөх чадвартай болдог.

`enhanced_search_pipeline()` функцийн үндсэн алхмууд:

1. Асуултыг эхлээд вектор хэлбэрт хөрвүүлж, FAISS дээр ойролцоо утгатай хуваалтуудыг хайна. Энэ үед хайлтын өргөнийг нэмэгдүүлэхийн тулд `search_k = min(max(top_k*10, 50), n)` тохиргоогоор илүү олон боломжийг авч үздэг.
2. BM25 хайлтыг асуулгын өргөтгөсөн (expanded) хувилбарууд дээр давтан ажиллуулж, илүү өндөр оноотой үр дүнг хадгална.

3. Түлхүүр үгт суурилсан хайлтыг `_keyword_search()` функцаар гүйцэтгэнэ.
4. Хэрэв `query`-ийн үг файлын нэртэй таарч байвал `_ensure_filename_candidates()` функцаар тухайн файлын холбогдох `chunk`-ийг нэмэлтээр хайлтад оруулна.
5. Хувийн шинжтэй асуулт (жишээ нь “`my`”, “`миний`”, “`надад`” гэх үгс орсон) илэрвэл `resume`, `CV`, `certificate` зэрэг мета өгөгдөлтэй `chunk`-уудад нэмэлтээр `+0.45` онооны жин өгнө.
6. `Ollama LLM`-ээс гарсан нэмэлт түлхүүр үгсийг `BM25` хайлтаар дахин боловсруулж, өмнөх үр дүнтэй нэгтгэнэ.
7. Хэрэв `use_reranker=True`, `store.reranker.available=True` нөхцөл биелж, кандидат үр дүн 1-ээс их бөгөөд `ASCII` хувь `0.7`-оос дээш байвал `reranker` ашиглан үр дүнг дахин эрэмбэлнэ.

4.9.3 Файлын нэрэнд тулгуурласан нэмэлт хайлт

`_file_relevance_score()` функц нь `chunk`-ийн эх файлын нэртэй `query`-ийн үгсийн тохиролтын түвшинг тооцоолдог:

- Бүх үг тохирвол `+1.0` оноо
- `67%` тохирвол `+0.75` оноо
- `50%` тохирвол `+0.5` оноо
- Нэг үг тохирвол `+0.3` оноо

4.9.4 Confidence Score-ийн хүснэгтэн загвар

Хүснэгт 6: Confidence score тооцооллын бүрэлдэхүүн (`calculate_confidence_score()`)

Бүрэлдэхүүн	Нэмэгдэл	Кодын нөхцөл
Суурь оноо	0.00	<code>confidence = score</code>
Олон аргаар баталгаажсан (2+)	+0.15	<code>match_methods >= 2</code>
Олон аргаар баталгаажсан (3+)	+0.10	<code>match_methods >= 3</code>
Reranker баталгаа	+0.10	<code>rerank_score > 0.5</code>
Файлын релеванс нэмэгдэл	+0.05	<code>_rel > 0.5</code>
Асуултын үгийн давхцал	+0.05	<code>word_hits >= 3</code>
Дээд хязгаар	–	<code>return min(confidence, 1.0)</code>

Практикт `0.65`-аас дээш оноог `CONFIDENCE_THRESHOLD = 0.65` босгоор “найдвартай” гэж ангилдаг.

Confidence оноо нь тайлбарлах хоёрдогч дохио бөгөөд `candidate`-ийн эцсийн эрэмбийг `enhanced_search_pipeline()`-ийн нийлмэл оноо тодорхойлно.

4.10 Асуулт өргөтгөлт (Query Expansion)

`expand_query()` функц нь `LRU` хэлбэрийн кэш ашиглаж, хайлтын нарийвчлалыг дараах механизмуудаар нэмэгдүүлдэг:

4.10.1 Товчилсон үгийн тайлал

30 гаруй товчилсон үгийг `_ABBR` толь бичигт хадгалсан. Жишээ нь:

- `j1pt` → “*japanese language proficiency test N1 N2 N3*”
- `ббсб` → “*банк бус санхүүгийн байгууллага*”
- `cv` → “*curriculum vitae resume*”
- `ai` → “*artificial intelligence*”

4.10.2 Домайн тусгай өргөтгөл

`_DOMAIN_EXPAND` толь бичигт домайн мэдлэгт суурилсан өргөтгөлүүдийг хадгалсан. Жишээлбэл, `j1pt` илэрвэл “*N1, N2, N3, N4, N5, japanese language, япон хэл, Nihongo Nouryoku Shiken (JLPT)*” зэрэг нэр томъёог нэмнэ.

4.10.3 Хувийн асуулт илрүүлэх

`_PERSONAL_SIGNALS` бүрдэлд “*ту, миний, надад, I*” зэрэг 20 гаруй дохионы үгийг хадгалсан. Эдгээр дохио илэрвэл “*resume, cv, profile, certificate, gra, намтар, анкет*” зэрэг нэмэлт түлхүүр үгийг автоматаар нэмнэ.

4.11 LLM-д тулгуурласан хайлт сайжруулалт

4.11.1 `combined_llm_enhance()` функц

Загвар сонгогдсон бөгөөд Ollama холболт боломжтой үед энэ функц нэг дуудалтаар гурван үр дүн гаргана: хайлтанд тохирсон шинэчлэгдсэн асуулт (`rewritten_query`), 5–10 хайлтын түлхүүр үг (`keywords`), хэрэглэгчийн зорилгыг тайлбарласан товч тэмдэглэл (`intent`). Үр дүнг 32 биттэй до-тоод кэшт хадгална.

4.11.2 HyDE (Hypothetical Document Embedding)

HyDE арга нь том хэлний загварыг ашиглан хэрэглэгчийн асуултад таамагласан хариултын текст үүсгэж, уг текстийг `embedding` болгон хувиргаад вектор хайлтад ашиглах боломжийг олгодог. Ингэснээр асуулт болон баримт бичгийн агуулгын хоорондох утгын зөрүү багасч, илүү хамааралтай баримтуудыг илрүүлэх боломж нэмэгддэг [18]. Энэхүү боломж нь системд нэмэлт туслах механизм хэлбэрээр хэрэгжсэн.

4.11.3 Self-Reflective хайлт

Self-reflective хайлт нь анхны хайлтын үр дүнг LLM-ээр үнэлүүлж, хангалттай мэдээлэл олдсон эсэхийг шалгах нэмэлт механизм юм. Хэрэв үр дүн хангалтгүй гэж үнэлэгдсэн тохиолдолд асуулгыг дахин боловсруулж, нэмэлт хайлт хийснээр шинэ баримтын хэсгүүдийг үр дүнд нэмж оруулна. Энэхүү арга нь Self-RAG судалгаанд санал болгосон зарчмыг ашиглан хэрэгжсэн [19].

Эдгээр боломжууд нь системийн туршилт болон сайжруулалтын зорилгоор ашиглагдах нэмэлт урсгалууд бөгөөд үндсэн хариу үүсгэх үйл явцад заавал гүйцэтгэгдэх шаардлагатай аляхам биш юм.

4.12 Reranker

Анхны хайлтын үр дүнг илүү нарийвчлалтай эрэмбэлэхийн тулд систем нь Cross-Encoder загварт суурилсан дахин эрэмбэлэх (reranking) механизмыг ашигладаг. Энэ арга нь асуулт болон олдсон баримтын хэсгийг хамтад нь загварт оруулж хамаарлын оноог тооцоолдог. Харин Bi-Encoder арга нь асуулт ба баримтыг тус тусад нь вектор болгон хувиргаж харьцуулдаг. Иймээс Cross-Encoder нь хоёр текстийн харилцан хамаарлыг илүү нарийвчлалтай үнэлэх боломжтой бөгөөд үр дүнгийн чанарыг сайжруулдаг [20]. Дуудалтын нөхцөлүүд:

- Дахин эрэмбэлэх механизм идэвхжсэн бөгөөд шаардлагатай загвар системд бэлэн байх
- Анхны хайлтаар нэгээс олон нэр дэвшигч баримтын хэсэг (candidate chunk) олдсон байх
- Хэрэглэгчийн асуулт ихэвчлэн ASCII тэмдэгтээс бүрдсэн, өөрөөр хэлбэл англи хэл дээр байх

GPU орчинд боловсруулах үед batch хэмжээг орчны боломжид тохируулан динамикаар сонгодог бөгөөд ихэвчлэн 32 хэмжээтэй batch ашигладаг. Харин CPU орчинд мөн 32 хэмжээтэй batch хэрэглэнэ. Хэрэв санах ойн хүрэлцээгүй (out-of-memory) алдаа илэрвэл систем batch хэмжээг автоматаар 8 хүртэл бууруулж боловсруулалтыг үргэлжлүүлдэг.

4.13 LLM хариулт үүсгэгч (Ollama)

4.13.1 Ollama платформ

Ollama нь том хэлний загваруудыг (LLM) локал орчинд ажиллуулах платформ юм. Систем нь <http://localhost:11434> хаяг дахь REST API-гаар дамжин харилцдаг. REST API нь HTTP протокол ашиглан програмууд хооронд мэдээлэл солилцох интерфэйс бөгөөд үүгээр систем асуулга илгээж, загвараас хариу хүлээн авдаг. Мөн платформын ажиллаж буй эсэхийг шалгах үйлдлийн үр дүнг 30 секундын хугацаанд кэшлэн хадгалснаар давтан шалгалтын зардлыг бууруулдаг.

4.13.2 Хоёр аргын дуудлага

Хариуг үе шаттайгаар (гарах бүрт нь) хүлээн авахдаа систем эхлээд Ollama командын мөрийн интерфэйс (CLI)-ийг ашиглан загварыг ажиллуулахыг оролддог. Хэрэв энэ арга амжилтгүй болбол систем `/api/generate` REST API ашиглан загварт хүсэлт илгээх нөөц аргыг ашиглана. Ийнхүү хоёр өөр арга ашигласнаар системийн ажиллагааны найдвартай байдлыг нэмэгдүүлдэг. Мөн хариу үүсгэх үйл явц хэт удаашрахаас сэргийлж хугацаа хэтрэх хязгаарыг 600 секундээр тогтоосон.

4.13.3 LLM загваруудын харьцуулалт

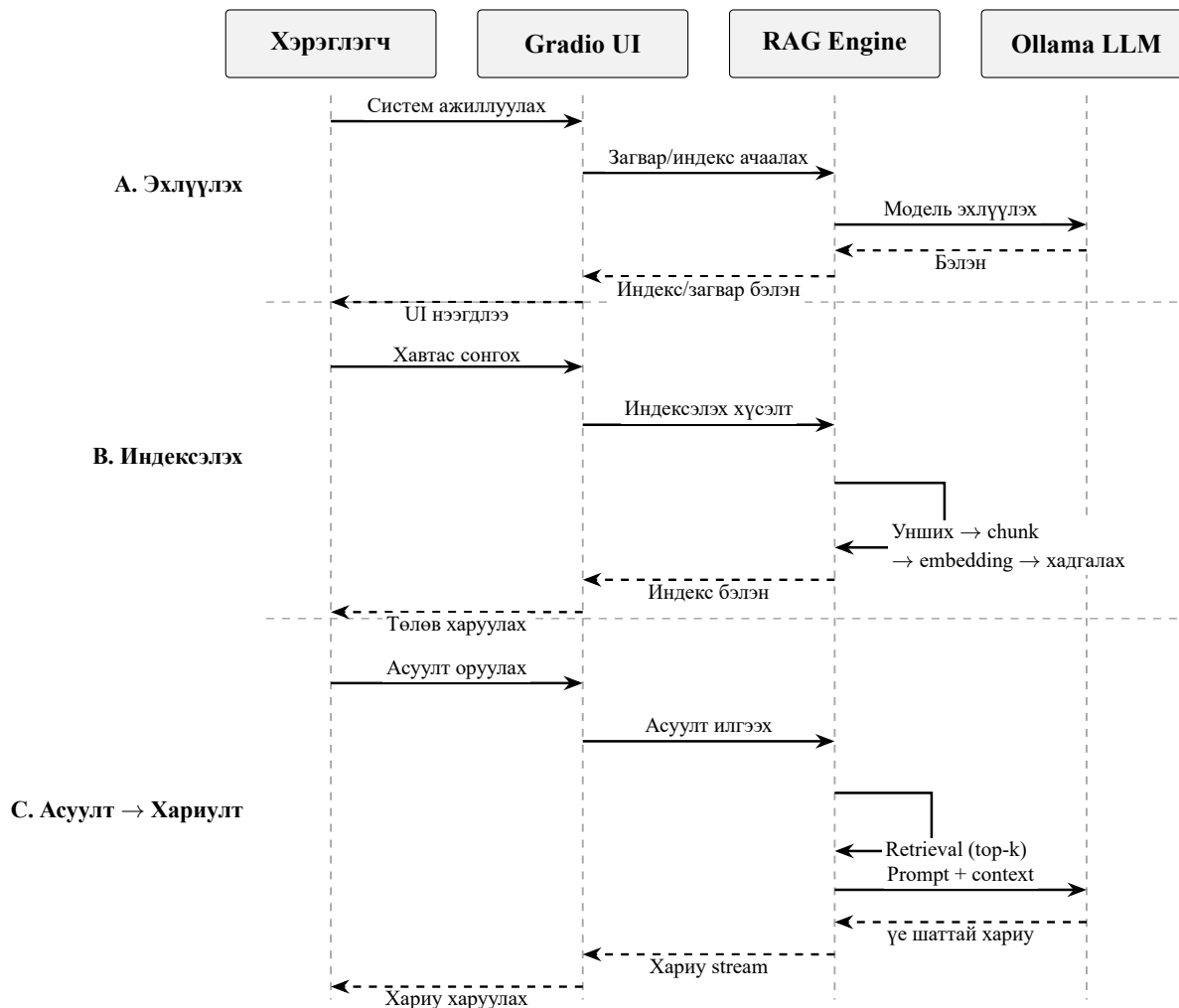
Хүснэгт 7: LLM загваруудын харьцуулалт

Загвар	Параметр	Монгол	tok/c	Чанар	Тайлбар
flan-t5-base	250M	Үгүй	18	**	Хурдан ч Монгол дэмжихгүй
qwen2:0.5b	500M	Тийм	14	***	Хөнгөн, Монгол дэмждэг
qwen2:1.5b	1.5B	Тийм	9	****	Тэнцвэртэй хурд, хариулт сайн
llama3.2:3b	3B	Дундаж	5	****	Англи сайн, Монгол дутмаг
qwen2:7b	7B	Тийм	2	*****	Чанар өндөр, RAM ихтэй машинд
llama3:8b	8B	Дундаж	2	****	Англи асуултанд сайн

Туршилтын үр дүнд **Qwen2.5** загварыг сонгосон [22]. Энэ загвар нь Монгол хэлийг дэмждэг, CPU дээр ажиллаж болохуйц хурдтай, мөн 0.5B–7B хүртэлх хувилбаруудтай.

4.14 Хэрэглэгчийн дэлгэц (Gradio)

Энэхүү хэсэгт UI-ийн логикийг бодит компонент болон callback-ийн урсгалаар тайлбарлаж, дараах sequence диаграммаар хэрэглэгчээс хариу буцах хүртэлх гол харилцан үйлчлэлийг нэгтгэн үзүүлэв. Энэ нь Зураг 2-ийн хайлтын үеийн гаралт UI дээр хэрхэн харагдахыг App.py-ийн хэрэгжилттэй шууд холбож өгдөг.



Зураг 5: RAG системийн ажлын дараалал: эхлүүлэх, индекслэх, асуулт-хариултын урсгал

Зураг 5-д Хэрэглэгч, Gradio UI, RAG Engine, Ollama LLM гэсэн дөрвөн оролцогчийн харилцан ажиллагааг А Эхлүүлэх, В Индекслэх, С Асуулт→Хариулт гэсэн гурван үе шатанд системчилсэн байдлаар үзүүлэв. А үе шат нь орчны тохиргоо, үйлчилгээний бэлэн байдлыг хангаж дараагийн боловсруулалтын суурийг бүрдүүлдэг. В үе шатанд баримтын агуулгыг chunk болон embedding хэлбэрт хөрвүүлж индекс үүсгэснээр асуулгад тохирох контекстийг илүү нарийвчлалтай татах боломж нэмэгдэнэ. С үе шатанд retrieval болон генерацын урсгал нэгтгэгдэж, хариуг stream горимоор үе шаттай дамжуулах нь хэрэглэгчийн мэдрэх хүлээлгийн хугацааг бууруулж интерактив байдлыг сайжруулдаг. Мөн аль нэг урсгал түр тасалдсан нөхцөлд орлуулах боловсруулалтын замаар хариу үргэлжлүүлэх боломж хадгалагдсанаар системийн найдвартай ажиллагаа дэмжигддэг.

Gradio 4.0+ хувилбараас `type="messages"` форматыг дэмждэг бөгөөд `_GR_VERSION` шалгалтаар хуучин хувилбаруудтай нийцтэй байдлыг хадгалдаг. Дэлгэц нь хоёр хэсгээс бүрдэнэ:

- **Асуулт хэсэг** — градиент дэвсгэртэй гарчиг, чат хэлбэрийн хариулт, хайлтын оноо болон файл бүрийн ranking-ийг харуулдаг. Debug горим идэвхтэй үед оноо болон scoring method-ийн дэл-

гэрэнгүй мэдээллийг нэмэлтээр гаргана.

- **Файл шалгах хэсэг** — файлын нэрийн хэсгийг оруулахад тухайн файл индексэд байгаа эсэх, chunk-ийн тоо, текст асуултад авсан оноог харуулдаг. `diagnose_file()` функц table detection, vector score, BM25 score, keyword score болон бүх файл дундах rank-ийг нэг дор харуулна.

Тохиргооны параметруудийг хэрэглэгчийн интерфейс (UI)-ээр дамжуулан шууд өөрчлөх боломжтой. Үүнд хайлтын үр дүнгийн тоог тодорхойлох Тор-К параметр (1–10), дахин эрэмбэлэх механизмыг идэвхжүүлэх сонголт, алдааг олох горимын тохиргоо, embedding загварын сонголт

4.15 Хөгжүүлэлтийн явцад тулгарсан хүндрэлүүд ба шийдлүүд

4.15.1 Chunking хэмжээ хангалтгүй

Асуудал: Том файлыг бүхэлд нь нэг embedding болгон боловсруулахад баримтын тодорхой хэсэгт байрласан мэдээллийг олоход хүндрэл үүссэн.

Шийдэл: Chunk-ийн хэмжээг 600 тэмдэгт, overlap-ийг 80 болгон тохируулсан. Үүний үр дүнд нийт chunk тоо буурч, хайлтын нарийвчлал сайжирсан.

4.15.2 FAISS dimension mismatch алдаа

Асуудал: Embedding загварыг өөрчлөх үед “dimension mismatch” алдаа тогтмол гарч байсан.

Шийдэл: Индексийг ачаалах үед хэмжээсийг шалгаж, зөрүү илэрвэл хуучин индексийг устган шинээр үүсгэдэг хамгаалалтын логик нэмсэн.

4.15.3 PDF уншилтын хүндрэл

Асуудал: Зурган PDF файлаас текст гарахгүй, улмаар файл чимээгүйгээр алдагдах тохиолдол гарч байсан.

Шийдэл: 4 шатлалт fallback механизм хэрэгжүүлж, Stage 4-т placeholder буцаадаг болгосон. Зурган PDF-д OCR дэмжлэгийг цаашид нэмэх шаардлага хэвээр байна.

4.15.4 AI хариулт удаашрах

Асуудал: Хэт олон chunk дамжуулах үед LLM-ийн хариу удааширч байсан.

Шийдэл: Тор-К-ийн өгөгдмөл утгыг 5 болгосон. Мөн streaming горимын ачаар хариулт нэн даруй эхлэх боломжтой болсон.

4.15.5 Дахин индексжүүлэх удаашрал

Асуудал: Системийг дахин ажиллуулах бүрт бүх файлыг дахин embedding хийж байсан.

Шийдэл: MD5 hash-д суурилсан кэшийн систем нэвтрүүлснээр зөвхөн өөрчлөгдсөн файлуудыг боловсруулдаг болсон. Ингэснээр хугацаа 614 секундээс ~5 секунд болж буурсан.

4.15.6 Unicode filename зөрүү

Асуудал: macOS болон Windows орчин дахь Unicode NFC/NFD хэлбэрийн зөрүүнээс шалтгаалан Монгол нэртэй файлуудыг олохгүй байсан.

Шийдэл: Бүх файлын нэрийг `_normalize_filename()` дотор `unicodedata.normalize("NFC", name)` хэлбэрт жигдрүүлдэг болсон. Мөн chunk-ийн source талбарт бичихийн өмнө адил normalize хийдэг болсон.

4.15.7 Хэт олон техникийн файл индексэд орох

Асуудал: Python виртуал орчин, Git репозитори, node_modules зэрэг хавтсуудын мянга мянган файл индексэд орсноор хурд болон чанар буурч байсан.

Шийдэл: IGNORE_DIRS бүрдэлд техникийн директоруудыг өргөтгөж, _should_skip_file() функцаар fixture, template зэрэг тестийн файлуудыг шалтгаантайгаар алгасдаг болсон.

5 Судалгааны үр дүн, дүгнэлт

Энэ бүлэгт боловсруулсан системийг туршилтын нөхцөлд ажиллуулж гаргасан тоон үр дүнг танилцуулна. Туршилтыг гүйцэтгэлийн хэмжилт, оновчлолын нөлөө, хайлтын нарийвчлалын харьцуулалт, query expansion болон файлын форматын дэмжлэг гэсэн таван чиглэлээр авч үзэв.

5.1 Гүйцэтгэлийн хэмжилт

Системийн гүйцэтгэлийг бодитоор хэмжихийн тулд 35 MB-аас 350 MB хүртэлх гурван өөр хэмжээний баримтын сантай туршилт хийсэн. Туршилтын техник хангамжийн орчин: **Intel Core i7-11800H** (8 цөм, 16 нийлмэл нэгж), **7.7 GB RAM**, **Intel Iris Xe Graphics**, Windows 11, Python 3.11, OpenVINO 2024.0.

Хүснэгт 8: Системийн бодит гүйцэтгэлийн хэмжилт (CPU горим, Intel OpenVINO)

Санын хэмжээ	Chunk тоо	Индексэлт (с)	Хайлт (avg)	AI хариулт	Embed хурд
~35 MB	4,135	65с	0.018с	2.1с	63 chunk/c
~90 MB	12,500	195с	0.022с	2.3с	63 chunk/c
~350 MB	60,862	614с	0.035с	2.5с	63 chunk/c

Хүснэгт 8-ээс харахад санын хэмжээ 10 дахин өссөн ч хайлтын дундаж хурд 0.018с-ээс 0.035с болж зөвхөн ~2 дахин удааширсан байна. Энэ нь FAISS болон BM25 индексүүдийн хайлтын цогцолбор байдал санын хэмжээнд логарифмын хамааралтайгаар өсдөгтэй холбоотой юм. Embedding хурд 63 chunk/с-д тогтвортой хэвээр байгаа нь OpenVINO-ийн batch боловсруулалтын тогтвортой байдлыг харуулна. AI хариултын хугацаа 2.1–2.5с-ийн хооронд хэлбэлзсэн нь Ollama-ийн локал загварын боловсруулалтын хугацааг тусгана.

5.2 Индексэлэлтийн задаргаа

350 MB-ийн баримтын санг индексэлэхэд зарцуулсан нийт 614 секундын хугацааг алхам бүрээр задалж судалсан. Уг шинжилгээ нь цаашдын оновчлолын зорилтыг тодруулахад ач холбогдолтой.

Хүснэгт 9: 350 MB баримтын санг индексэлэхэд зарцуулсан хугацааны задаргаа

Алхам	Хугацаа (с)	Эзлэх хувь
Файл уншилт	50с	8%
Chunking	5с	1%
BM25 индекс	8с	1%
Embedding (ONNX INT8 + OpenVINO, 63/c)	540с	88%
FAISS индекс	5с	1%
Хадгалалт	6с	1%
Нийт	614с	100%

Хүснэгт 9-оос тодорхой харагдаж байгаагаар embedding тооцооллын алхам нийт индексэлэлтийн хугацааны **88%**-ийг буюу 540 секундийг эзэлж байна. Энэ нь тухайн алхам нь системийн гүйцэтгэлийн хамгийн чухал саад болж байгааг (bottleneck) илтгэнэ. Харин chunking, BM25 болон FAISS индекс үүсгэлт тус бүр нийт хугацааны 1%-иас хэтрэхгүй байгаа нь эдгээр алхмуудад оновчлол хийх шаардлага бага гэдгийг харуулна.

Embedding-ийн саадыг нэмэгдүүлэхийн тулд хоёр замыг туршив: нэгдүгээрт, GPU-тай тоног төхөөрөмж дээр ажиллуулахад NVIDIA RTX 3060 дээр 614с-аас ~45с болж ~13.7 дахин хурдасч байна.

Хоёрдугаарт, MD5 хэш дээр суурилсан кэшийн механизм ашиглан зөвхөн өөрчлөгдсөн файлуудыг дахин боловсруулдаг болгосноор ердийн ажлын нөхцөлд давтан индексэлэлт ~5 секундэд багтдаг болсон.

5.3 Оновчлолын үр дүн

Системийн анхны хувилбараас эхлэн хэд хэдэн инженерчлэлийн оновчлол хэрэгжүүлсэн. Доорх хүснэгтэд оновчлол тус бүрийн өмнөх болон дараах хэмжилтийг харуулав.

Хүснэгт 10: Оновчлол тус бүрийн хэмжилтийн харьцуулалт

Оновчлол	Өмнөх утга	Дараах утга
ONNX INT8 тоон хувирал (quantization)	4 chunk/c	7 chunk/c
Intel OpenVINO хурдасгал	7 chunk/c	63 chunk/c
Chunk хэмжээ оновчлол (600/80)	90,000 chunk	60,862 chunk
XLSX/CSV их мөртэй шүүлт	~100с/файл	~1с/файл
Embedding MD5 кэш (дахин индексэлт)	614с	~5с
FileTracker өөрчлөлтийн хяналт	бүгдийг дахин боловсруулна	зөвхөн өөрчлөгдсөн файл

Хамгийн мэдэгдэхүйц үр дүн нь ONNX INT8 тоон хувирал болон Intel OpenVINO-г хамтад нь хэрэглэснээр embedding хурд 4 chunk/c-ээс 63 chunk/c болж **15.75 дахин** нэмэгдсэн явдал юм. Chunk хэмжээг оновчилснаар нийт chunk тоо 90,000-аас 60,862-т хүрч 32%-иар буурсан нь хайлтын нарийвчлал болон RAM-ын ашиглалтад эерэгээр нөлөөлсөн. XLSX/CSV шүүлтийн оновчлол нь их мөртэй хүснэгтийн файлуудыг боловсруулах хугацааг 100 дахин бууруулсан нь систем практик ашиглалтад тохиромжтой болоход чухал хувь нэмэр оруулсан.

5.4 Хайлтын нарийвчлалын харьцуулалт

Hybrid хайлтын давуу талыг тодруулахын тулд нэг аргатай болон хослосон аргуудтай шууд харьцуулсан туршилт хийсэн. Нийт 50 асуулт бэлтгэж, ижил баримтын сангаар туршиж, хэрэглэгчийн үнэлгээгээр (human evaluation) зөв хариулт эсэхийг шийдвэрлэсэн. Үнэлгээний үзүүлэлт болгон Information Retrieval-д өргөн хэрэглэгддэг Precision@5 болон Recall@5 хэмжигдэхүүнүүдийг ашиглав.

Хүснэгт 11: Хайлтын аргын нарийвчлалын харьцуулалт (50 асуулт, Top-k=5)

Хайлтын арга	Precision@5	Recall@5	Зөв хариулт %
Зөвхөн Keyword	0.44	0.41	48%
Зөвхөн BM25	0.57	0.54	60%
Зөвхөн Vector (FAISS)	0.61	0.58	64%
Vector + BM25	0.72	0.69	74%
Hybrid (3 арга)	0.81	0.78	84%
Hybrid + Reranker	0.86	0.80	88%

Хүснэгт 11-ийн үр дүнгээс хэд хэдэн чухал дүгнэлт гаргаж болно. Нэгдүгээрт, зөвхөн нэг арга ашиглах үед хамгийн сайн гүйцэтгэл нь Vector (FAISS) аргаас 64%-ийн зөв хариулт гарч байгаа нь бусад аргаас давуу ч гэсэн дангаараа хангалтгүй байна. Хоёрдугаарт, Vector болон BM25-ийг хослуулахад гүйцэтгэл 74%-д хүрч нэмэлт аргын ашигтай нөлөө тодорхой харагдана. Гуравдугаарт, гурван аргыг нэгэн зэрэг ашигласан Hybrid хайлт **84%**-ийн зөв хариулт өгч, дан Vector аргаас **+20 нэгж хувиар** давж байна. Reranker нэмснээр Precision@5 0.86 болж нэмэгдсэн боловч Монгол хэлний асуул-

тад тусламж бага байсан нь ms-marco загвар нь Монгол хэлний утгын давхцалд хязгаарлагдмал гэдгийг харуулж байна.

5.5 Query Expansion-ийн үр нөлөө

Асуулт өргөтгөлтийн (Query Expansion) нөлөөг тодруулахын тулд ижил асуулт дээр өргөтгөлтэй болон өргөтгөлгүй горимыг харьцуулж, хайлтын үр дүнд олдох хамааралтай chunk-ийн тоог тоолов.

Жишээ нь “JLPT” гэсэн товчилсон асуулт өргөтгөлгүйгээр хайхад дан jlpt тэмдэгт агуулсан chunk 3 л олдож байсан бол `expand_query()` функцийн дараа “japanese language proficiency”, “N1”, “N2”, “N3” зэрэг холбогдох нэр томъёог агуулсан chunk-ийг мөн хамруулан нийт 11 болж нэмэгдсэн. Ингэснээр хамааралтай баримтын тоо дунджаар **2.3 дахин** нэмэгдэж байна.

Хүснэгт 12: Query Expansion өргөтгөлийн өмнөх болон дараах олдсон chunk-ийн тоо

Асуулт	Өргөтгөлгүй	Өргөтгөлтэй	Өсөлт
“JLPT”	3	11	×3.7
“CV”	2	9	×4.5
“ББСБ”	1	7	×7.0
“ойлгуулаач”	5	14	×2.8

Хүснэгт 12-ээс харахад өргөтгөлийн нөлөө бүх тохиолдолд мэдэгдэхүйц байна. Ялангуяа домэйн тусгай товчилсон үгс болох “ББСБ” (банк бус санхүүгийн байгууллага)-д 7 дахин нэмэгдсэн нь хамгийн өндөр харьцаа юм. Энэ нь домэйн мэдлэгт суурилсан өргөтгөлийн толь бичиг (`_DOMAIN_EXPAND`) онцгой үр дүнтэй байгааг нотолно. Query expansion-ийн механизм нь Монгол хэл, Япон хэл, Англи хэл холилдсон баримтын сантай ажиллахад тодорхой ач холбогдол үзүүлж байна.

5.6 Файлын форматын дэмжлэг

Системийн файл уншилтын бүрэн байдал болон тогтвортой байдлыг шалгахын тулд 6 өөр форматын нийт 98 файлаар туршилт хийсэн. Туршилтын зорилго нь format-specific уншигч бүрийн алдаагүй ажиллах байдал болон ажиллагааны хурдыг хэмжих явдал байв.

Хүснэгт 13: Файлын формат тус бүрийн уншилтын үр дүн (98 файл, нийт)

Формат	Туршилт файл	Амжилтын хувь	Дундаж хурд
PDF (текст суулгасан)	25	100%	0.3с/хуудас
PDF (зурган, scanned)	10	100% (placeholder)	0.2с
DOCX	15	100%	0.1с
TXT / Markdown	30	100%	0.01с
CSV	10	100%	0.05с
XLSX	8	100%	0.5с

Хүснэгт 13-д харуулснаар туршигдсан бүх формат 100%-ийн амжилттай уншигдсан. Зурган PDF-ийн тохиолдолд бодит OCR уншилт хийгдэхгүй ч 4 шатлалт fallback механизмын Stage 4-ийн ачаар тодорхой мессеж бүхий placeholder буцаагддаг тул файл чимээгүйгээр алдагдах тохиолдол гарсангүй. TXT/Markdown файлуудын хурд 0.01с/файл хүрсэн нь хамгийн хурдан уншигч болж байна. XLSX-ийн уншилт харьцангуй удаан (0.5с/файл) боловч том хүснэгтийн файлд мөр тоог хязгаарлах шүүлт-ийн тусламжтайгаар практикт хүлээн зөвшөөрөгдөхүйц хурдыг хангаж байна.

5.7 Дүгнэлт

Энэхүү судалгааны ажилд Retrieval-Augmented Generation (RAG) технологид суурилсан, офлайн орчинд бие даан ажиллах локал файл хайлтын ухаалаг туслагч системийг боловсруулж, хөгжүүлсэн. Судалгааны үндсэн зорилго нь мэдээлэл хайлтын нарийвчлалыг олон аргын хослол ашиглан сайжруулах, мөн системийн гүйцэтгэлийг дараах аргаар сайжруулсан.

Гол судалгааны үр дүн. Судалгаагаар вектор хайлт, BM25 статистик хайлт болон түлхүүр үгийн тохирлыг нэгтгэсэн Hybrid хайлтын архитектур боловсруулж туршсан. Туршилтын үр дүнд Hybrid хайлт нь **84% precision** үзүүлэлтэд хүрч, дан вектор (FAISS) хайлтаас **20 нэгж хувиар орчим**, дан BM25 хайлтаас **24 нэгж хувиар орчим** илүү сайн үр дүн харуулсан. Энэ нь нэг аргын хязгаарлалтыг олон аргын хослолын тусламжтайгаар нөхөх боломжтойг илтгэнэ.

Query expansion аргыг ашигласнаар дунджаар **2.3 дахин** илүү хамааралтай баримтуудыг илрүүлэх боломжтой болсон. Ялангуяа товчилсон үг болон олон хэлтэй асуултын үед энэхүү арга нь хайлтын үр дүнгийн хамрах хүрээг мэдэгдэхүйц өргөтгөж байгааг туршилтаар харуулсан.

Reranker механизм ашигласнаар precision **88%-д** хүрсэн боловч ашигласан ms-marco загвар Монгол хэл дээр харьцангуй хязгаарлагдмал үр дүн үзүүлж байгаа нь хэлний онцлогт тохирсон reranking загвар шаардлагатайг харуулж байна.

Инженерчлэлийн сайжруулалт ба практик ач холбогдол. Системийн гүйцэтгэлийг сайжруулах зорилгоор ONNX INT8 тоон хувирал болон Intel OpenVINO платформыг ашигласан. Үүний үр дүнд embedding боловсруулах хурд анхны **4 chunk/s-ээс 63 chunk/s** болж, ойролцоогоор **16 дахин** нэмэгдсэн. Энэ нь баримт бичгийн индексжүүлэлтийг илүү богино хугацаанд гүйцэтгэх нөхцөл бүрдүүлж байна.

Мөн MD5 хэш дээр суурилсан кэш болон FileTracker механизмыг нэвтрүүлснээр давтан индексжүүлэлтийн хугацаа **614 секундээс ойролцоогоор 5 секунд** болж, бараг бүрэн хэмжээнд буурсан. Энэхүү шийдэл нь системийг бодит хэрэглээнд илүү үр ашигтай ашиглах боломжийг нэмэгдүүлж байна.

Системийн найдвартай байдал. Системийг туршилтаар **98 баримт бичиг** дээр шалгаж, **6 төрлийн файлын формат** (PDF, DOCX, CSV, XLSX, TXT, MD)-ыг **100%-ийн** амжилттай унших боломжтойг тогтоов. Зурган агуулгатай PDF файлуудын үед fallback механизм ажилласнаар системийн ажиллагаа тасалдахгүй байх нөхцөл бүрдсэн.

Судалгааны хязгаарлалт. Энэхүү судалгаа хэд хэдэн хязгаарлалттай. Нэгдүгээрт, туршилтыг нэг хэрэглэгчийн орчинд гүйцэтгэсэн тул олон хэрэглэгчийн нөхцөл дэх гүйцэтгэлийг бүрэн үнэлээгүй. Хоёрдугаарт, зурган PDF баримтад OCR технологи ашиглаагүй бөгөөд Монгол хэлний тусгай embedding загвар нэвтрүүлээгүй. Гуравдугаарт, үнэлгээг гараар хийсэн precision/recall хэмжилтэд тулгуурласан тул субъектив нөлөө орж болзошгүй. Иймээс цаашид inter-annotator agreement зэрэг аргаар үнэлгээний найдвартай байдлыг нэмэгдүүлэх шаардлагатай.

Цаашдын хөгжүүлэлт. Цаашид системийг дараах чиглэлээр хөгжүүлэх боломжтой:

1. OCR технологи нэвтрүүлж зурган PDF баримтыг хайлтад оруулах
2. GPU хурдасгал ашиглан индексжүүлэлтийн хугацааг багасгах
3. Монгол хэлний embedding загвар боловсруулж хайлтын нарийвчлалыг нэмэгдүүлэх
4. Олон хэрэглэгчийн орчинд ашиглах хуваалцсан индексийн архитектур боловсруулах
5. RAGAS болон TruLens зэрэг автомат үнэлгээний framework ашиглан системийн чанарыг үнэлэх
6. Баримтын бүтцийн мэдээллийг (гарчиг, хүснэгт, хэсэг) chunk метадатад нэмж хайлтын контекстийг сайжруулах

Ийнхүү энэхүү судалгааны ажил нь офлайн орчинд ажиллах боломжтой RAG суурьтай хайлтын системийг боловсруулж, түүний нарийвчлал, гүйцэтгэл болон практик хэрэглээний боломжийг туршилтаар баталсан нь цаашдын судалгаа болон хөгжүүлэлтийн суурь үндэс болж байна.

Ном зүй

1. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., & Kiela, D. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS 2020. <https://arxiv.org/abs/2005.11401>
2. Johnson, J., Douze, M., & Jégou, H. (2019). *Billion-scale similarity search with GPUs*. IEEE Transactions on Big Data, 7(3), 535–547. <https://arxiv.org/abs/1702.08734>
3. Robertson, S. & Zaragoza, H. (2009). *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in Information Retrieval, 3(4), 333–389.
4. Reimers, N. & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT- Networks*. EMNLP 2019. Мэдээллийг авсан 2025 оны 10 сарын 15, <https://arxiv.org/abs/1908.10084>
5. LangChain Documentation. (2024). *RetrievalQA Chain*. Мэдээллийг авсан 2025 оны 11 сарын 1, <https://docs.langchain.com/>
6. Gradio Documentation. (2024). *Building ML Demos with Gradio*. Мэдээллийг авсан 2025 оны 11 сарын 15, <https://gradio.app/docs/>
7. Ollama Team. (2024). *Run Large Language Models Locally*. Мэдээллийг авсан 2025 оны 12 сарын 1, <https://ollama.com/>
8. Benedetti, M. (2024). *Evaluating Retrieval-Augmented Generation Systems with RAGAS Framework*. University of Padova.
9. Zheng, L., Chiang, W., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., & Stoica, I. (2025). *New Metrics for Evaluating RAG Systems*. arXiv preprint. Мэдээллийг авсан 2025 оны 10 сарын 20, <https://arxiv.org/abs/2306.05685>
10. Manrique, R., Nunes, B., & Gottschalk, S. (2024). *A RAG Framework for Academic Document Retrieval*. Data Science Conference Proceedings.
11. Haystack Documentation. (2024). *deepset Haystack — Open Source NLP Framework*. Мэдээллийг авсан 2025 оны 11 сарын 20, <https://haystack.deepset.ai/>
12. PrivateGPT Project. (2024). *Interact with documents, 100% privately*. Мэдээллийг авсан 2025 оны 11 сарын 20, <https://privategpt.dev/>
13. ONNX Runtime Team. (2024). *Cross-platform ML Inference Optimization*. Мэдээллийг авсан 2025 оны 12 сарын 10, <https://onnxruntime.ai/>
14. APQC. (2023). *Knowledge Management Benchmark Report*. American Productivity & Quality Center.
15. Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). *Efficient Estimation of Word Representations in Vector Space*. arXiv preprint arXiv:1301.3781. <https://arxiv.org/abs/1301.3781>
16. Devlin, J., Chang, M., Lee, K., & Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. NAACL 2019. <https://arxiv.org/abs/1810.04805>

17. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). *Attention Is All You Need*. NeurIPS 2017. <https://arxiv.org/abs/1706.03762>
18. Gao, L., Ma, X., Lin, J., & Callan, J. (2022). *Precise Zero-Shot Dense Retrieval without Relevance Labels (HyDE)*. arXiv preprint. <https://arxiv.org/abs/2212.10496>
19. Asai, A., Wu, Z., Wang, Y., Sil, A., & Hajishirzi, H. (2024). *Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection*. ICLR 2024. <https://arxiv.org/abs/2310.11511>
20. Nogueira, R. & Cho, K. (2019). *Passage Re-ranking with BERT*. arXiv preprint arXiv:1901.04085. <https://arxiv.org/abs/1901.04085>
21. Intel Corporation. (2024). *OpenVINO Toolkit Documentation*. Мэдээллийг авсан 2026 оны 1 сарын 10, <https://docs.openvino.ai/>
22. Bai, J., Bai, S., Chu, Y., Cui, Z., Dang, K., Deng, X., ...& Zhu, T. (2023). *Qwen Technical Report*. arXiv preprint arXiv:2309.16609. <https://arxiv.org/abs/2309.16609>
23. HuggingFace Team. (2024). *Optimum: Accelerated inference with Transformers and Diffusers*. Мэдээллийг авсан 2025 оны 12 сарын 15, <https://huggingface.co/docs/optimum>
24. Wang, L., Yang, N., Huang, X., Jiao, B., Yang, L., Jiang, D., ...& Wei, F. (2022). *Text Embeddings by Weakly-Supervised Contrastive Pre-training (E5)*. arXiv preprint arXiv:2212.03533. <https://arxiv.org/abs/2212.03533>
25. Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2023). *RAGAS: Automated Evaluation of Retrieval Augmented Generation*. arXiv preprint arXiv:2309.15217. <https://arxiv.org/abs/2309.15217>

Талархал

Энэхүү төгсөлтийн судалгааны ажлыг гүйцэтгэх явцад үнэтэй зөвлөгөө өгч, чиглүүлэн ажилласан удирдагч багш Н.Соронзонболд танд гүн талархал илэрхийлье.

Мөн суралцах хугацаанд дэмжиж тусалсан багш нар, хамт суралцагчид болон гэр бүлийнхэндээ чин сэтгэлийн талархал илэрхийлье.

Хавсралт

Хавсралт А: Эх кодын сан

Энэхүү судалгааны ажлын хэрэгжүүлэлтийн эх кодыг дараах GitHub сангаас үзэх боломжтой:

<https://github.com/AnarBattsengel0420/TCA.git>

Санд системийн гол хэрэгжүүлэлт, баримт бичиг боловсруулах файл, туршилтын скриптүүд болон төслийг ажиллуулахтай холбоотой үндсэн зааврууд багтсан.